ADVENTIST UNIVERSITY OF CENTRAL AFRICA (AUCA)

Course

Advanced Big Data Analytics

Project Title

Mobile Money Investigations with PostgreSQL and Neo4j

Assignment II

Group 17

Student ID	Member Name
101380	Fulgence Muhirwa
101195	Uwineza Deborah
101385	Babatunde Oyedele

Date Due

October 26, 2026, 11:59 PM CAT

GitHub repository

https://github.com/Mfulgence/Mobile_Money_Investigations_with_PostgreSQL_and_Neo4j

Table of Contents

Mobile Money Investigations with PostgreSQL and Neo4j 3
1 Capture the data 3
1.1 Workspace 3
1.2 capture.py 4
2 PostgreSQL 6
2.1 Schema 6
2.2 Load and checks 7
3 Neo4j 9
3.1 Graph mapping 9
3.2 Import 10
3.3 One account in both models 15
4 Routine business questions 15
4.1 Q1 Account activity 16
4.2 Q2 Daily activity 17
4.3 Q3 Shared devices 18
5 Connected accounts 19
5.1 Q4 Ownership and device network 19
5.2 Q5 Rapid onward payments 21
5.3 Q6 Circular payments 22
5.4 Q7 Shortest transfer connection (person accounts only) 23
6 Correctness and performance 24
6.1 Comparison program 24
6.2 Timings (3 warm-ups, 5 measured runs, one engine at a time) 26
6.3 SQL improvement for Q7 28
7 Concurrent requests 29
7.1 Load test 29
7.2 Scaling 31
8 Decisions 32
8.1 Investigation priorities 32
8.2 Recommendation (≤ 500 words) 33
8.3 Individual judgement 34
9 Reproducibility 36

Mobile Money Investigations with PostgreSQL and Neo4j

Models compared: relational (PostgreSQL 18.6) and property graph (Neo4j 2026.10.1 Community), on one laptop, using the fixed September 2026 dataset (20,993,500 records) from mobile_money_generator.py.

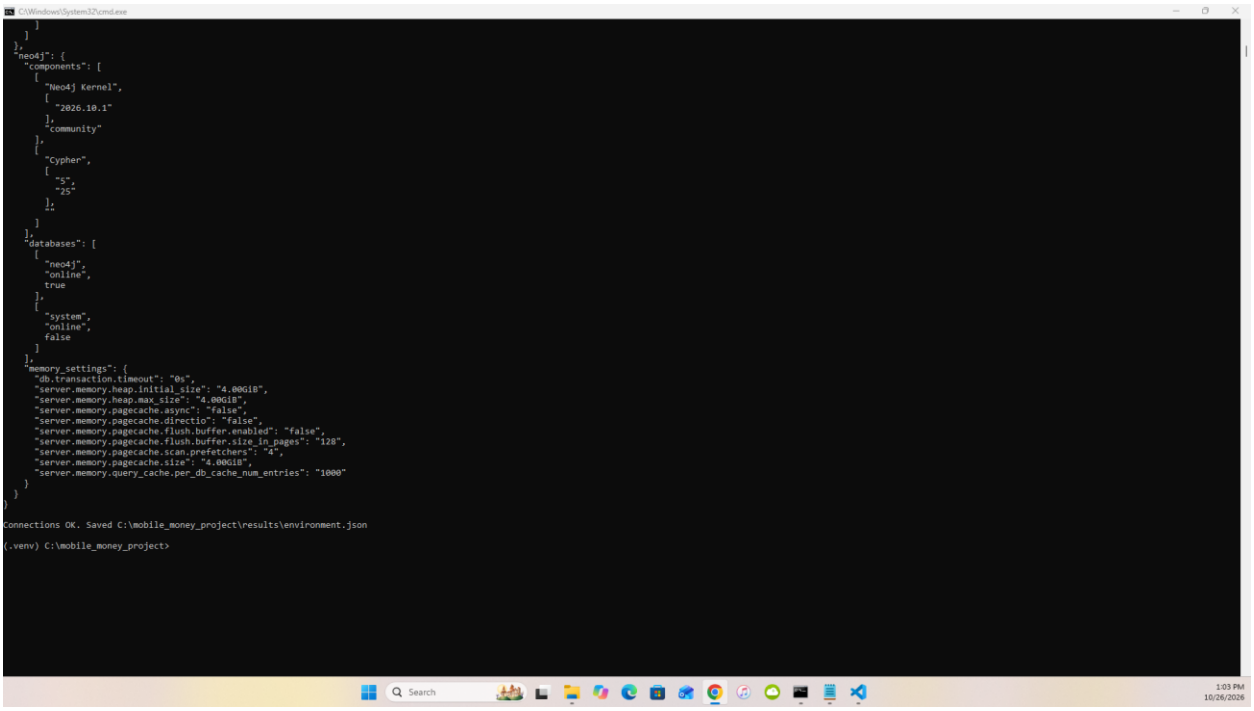
---

1 Capture the data

1.1 Workspace

All work ran on one laptop. step01_environment.py recorded the versions, hardware and database memory settings below (results/environment.json) and proved both connections work. Two PostgreSQL versions were installed (16 and 18); version 18 owns port 5432 and was used throughout, and the unused 16 service was stopped and set to manual so it could not affect timings.

Item	Value
OS / CPU / RAM	Windows 11 (build 26300) · Intel Core i7-10510U, 4 cores / 8 threads, 1.8 GHz · 16 GB
Python / psycopg / neo4j driver / numpy	3.12.6 / 3.3.6 / 6.4.1 / 2.5.4
PostgreSQL	18.6 (x86_64-windows); shared_buffers 4 GB, effective_cache_size 8 GB, work_mem 64 MB, maintenance_work_mem 1 GB, max_wal_size 8 GB; TimeZone = Africa/Kigali (server and database)
Neo4j	2026.10.1 Community, Cypher 5/25, on Temurin Java 21.0.12 (used only for Neo4j; system Java stays 17); heap 4 GB initial/max, page cache 4 GB



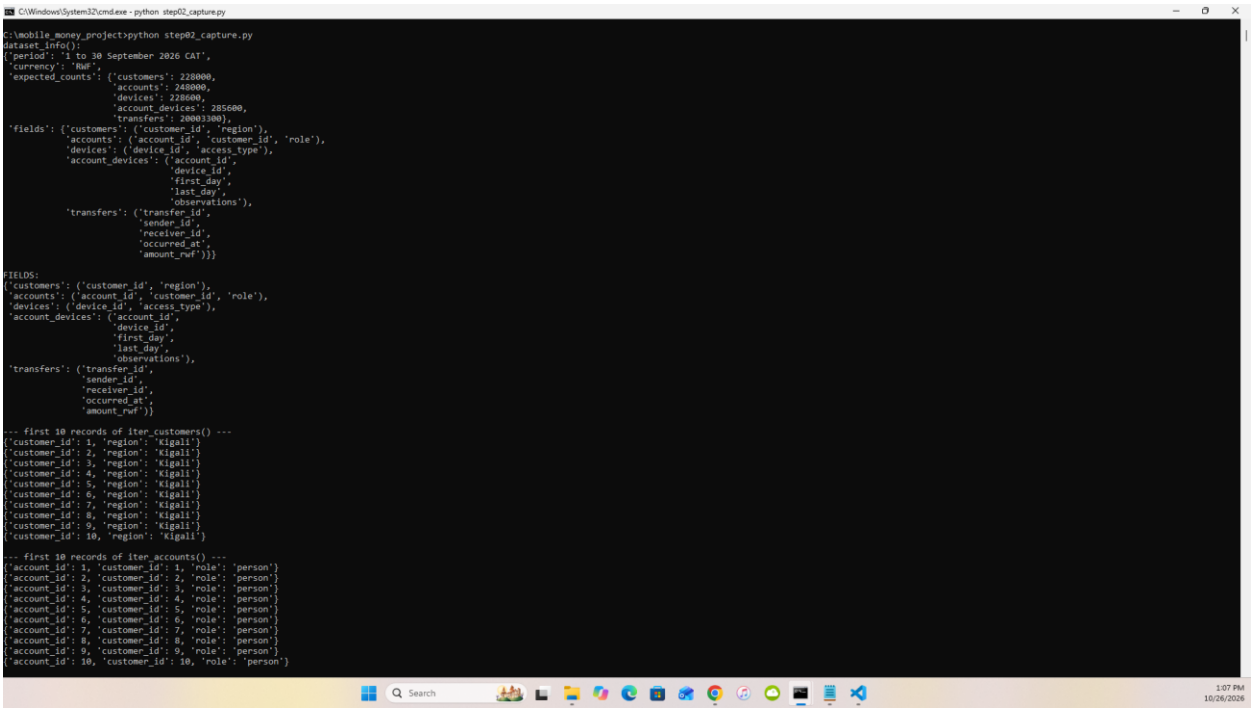
1.1 – step01 output: both connections, TimeZone, memory settings

1.2 capture.py

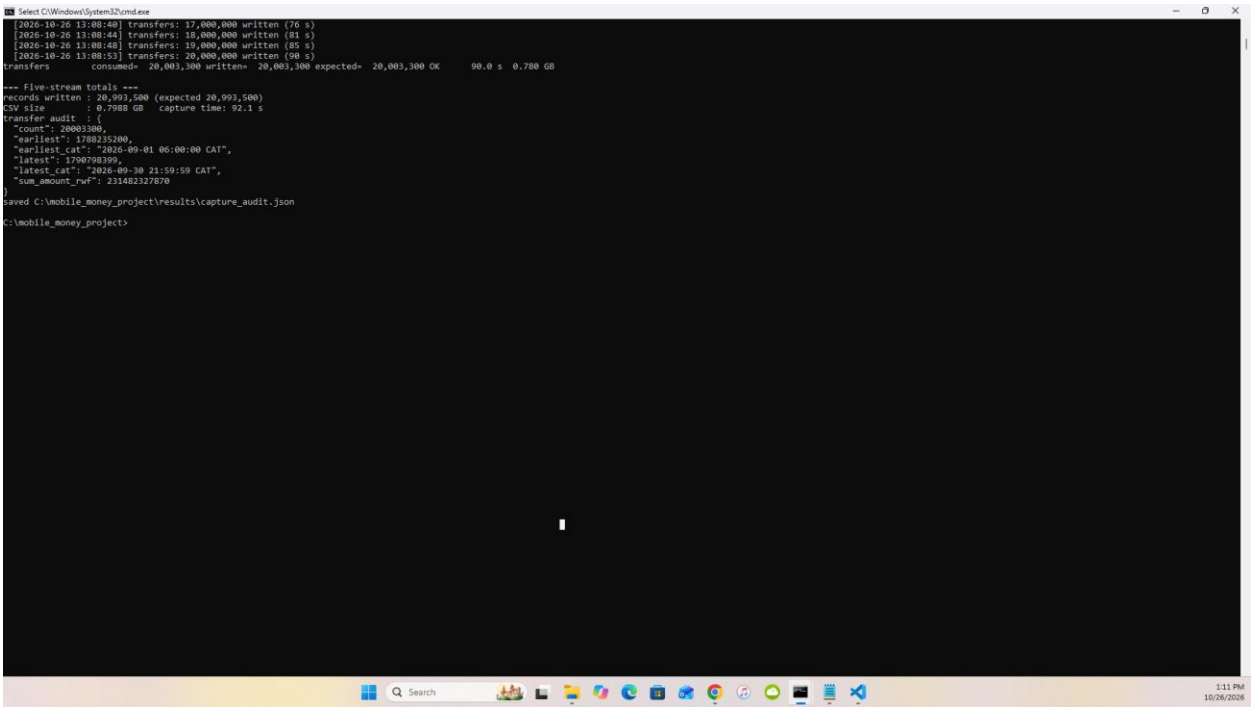
step02_capture.py previews ten records per stream with itertools.islice, then streams each generator straight into csv.DictWriter (no stream is held in memory), counting consumed and written rows and logging every million transfers. Because transfers are **not time-sorted**, the earliest/latest occurred_at are tracked as running min/max.

Stream	Rows written	Expected	Seconds	CSV GB
customers	228,000	228,000	0.37	0.004
accounts	248,000	248,000	0.53	0.005
devices	228,600	228,600	0.37	0.004
account_devices	285,600	285,600	0.78	0.006
transfers	20,003,300	20,003,300	90.01	0.780
Total	**20,993,500**	20,993,500	**92.1**	**0.799**

The full capture took **92.1 s**, almost all of it the 20 M transfers (90.0 s, ≈222,000 rows/s), writing 0.80 GB of CSV. Transfer audit: count 20,003,300 · earliest 1,788,235,200 (2026-09-01 06:00:00 CAT) · latest 1,790,798,399 (2026-09-30 21:59:59 CAT) · sum 231,482,327,870 RWF.


C:\Windows\System32\cmd.exe - python step02_capture.py
C:\mobile_money_project>python step02_capture.py
dataset_info():
{'period': '1 to 30 September 2026 CAT',
'currency': 'RWF',
'expected_counts': {'customers': 228000,
'accounts': 248000,
'devices': 228600,
'account_devices': 285600,
'transfers': 20003300},
'fields': {'customers': ('customer_id', 'region'),
'accounts': ('account_id', 'customer_id', 'role'),
'devices': ('device_id', 'access_type'),
'account_devices': ('account_id',
'device_id',
'first_day',
'last_day',
'observations'),
'transfers': ('transfer_id',
'sender_id',
'receiver_id',
'occurred_at',
'amount_rwf')}}
FIELDS:
{'customers': ('customer_id', 'region'),
'accounts': ('account_id', 'customer_id', 'role'),
'devices': ('device_id', 'access_type'),
'account_devices': ('account_id',
'device_id',
'first_day',
'last_day',
'observations'),
'transfers': ('transfer_id',
'sender_id',
'receiver_id',
'occurred_at',
'amount_rwf')}
-- first 10 records of iter_customers() ---
{'customer_id': 1, 'region': 'Kigali'}
{'customer_id': 2, 'region': 'Kigali'}
{'customer_id': 3, 'region': 'Kigali'}
{'customer_id': 4, 'region': 'Kigali'}
{'customer_id': 5, 'region': 'Kigali'}
{'customer_id': 6, 'region': 'Kigali'}
{'customer_id': 7, 'region': 'Kigali'}
{'customer_id': 8, 'region': 'Kigali'}
{'customer_id': 9, 'region': 'Kigali'}
{'customer_id': 10, 'region': 'Kigali'}
-- first 10 records of iter_accounts() ---
{'account_id': 1, 'customer_id': 1, 'role': 'person'}
{'account_id': 2, 'customer_id': 2, 'role': 'person'}
{'account_id': 3, 'customer_id': 3, 'role': 'person'}
{'account_id': 4, 'customer_id': 4, 'role': 'person'}
{'account_id': 5, 'customer_id': 5, 'role': 'person'}
{'account_id': 6, 'customer_id': 6, 'role': 'person'}
{'account_id': 7, 'customer_id': 7, 'role': 'person'}
{'account_id': 8, 'customer_id': 8, 'role': 'person'}
{'account_id': 9, 'customer_id': 9, 'role': 'person'}
{'account_id': 10, 'customer_id': 10, 'role': 'person'}
Search
1:07 PM
10/26/2026

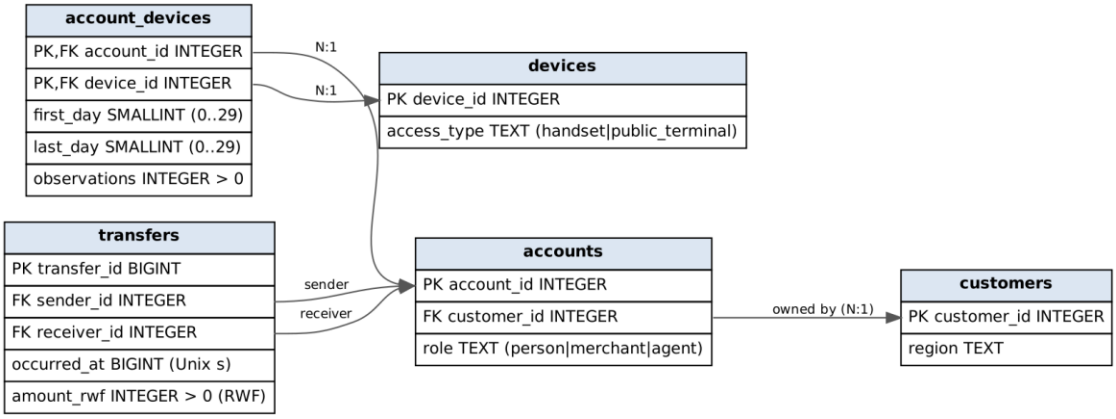
1.2a


Select C:\Windows\System32\cmd.exe
[2026-10-26 13:08:40] transfers: 17,000,000 written (76 s)
[2026-10-26 13:08:44] transfers: 18,000,000 written (81 s)
[2026-10-26 13:08:48] transfers: 19,000,000 written (85 s)
[2026-10-26 13:08:53] transfers: 20,000,000 written (90 s)
transfers consumed= 20,003,300 written= 20,003,300 expected= 20,003,300 OK 90.0 s 0.780 GB
--- Five-stream totals ---
records written : 20,993,500 (expected 20,993,500)
CSV size : 0.7988 GB capture time: 92.1 s
transfer_audit : {
"count": 20003300,
"earliest": 1788235200,
"earliest_cat": "2026-09-01 06:00:00 CAT",
"latest": 1790790399,
"latest_cat": "2026-09-30 21:59:59 CAT",
"sum_amount_rwf": 231482327870
}
saved C:\mobile_money_project\results\capture_audit.json
C:\mobile_money_project>
Search
1:11 PM
10/26/2026

1.2b

2 PostgreSQL

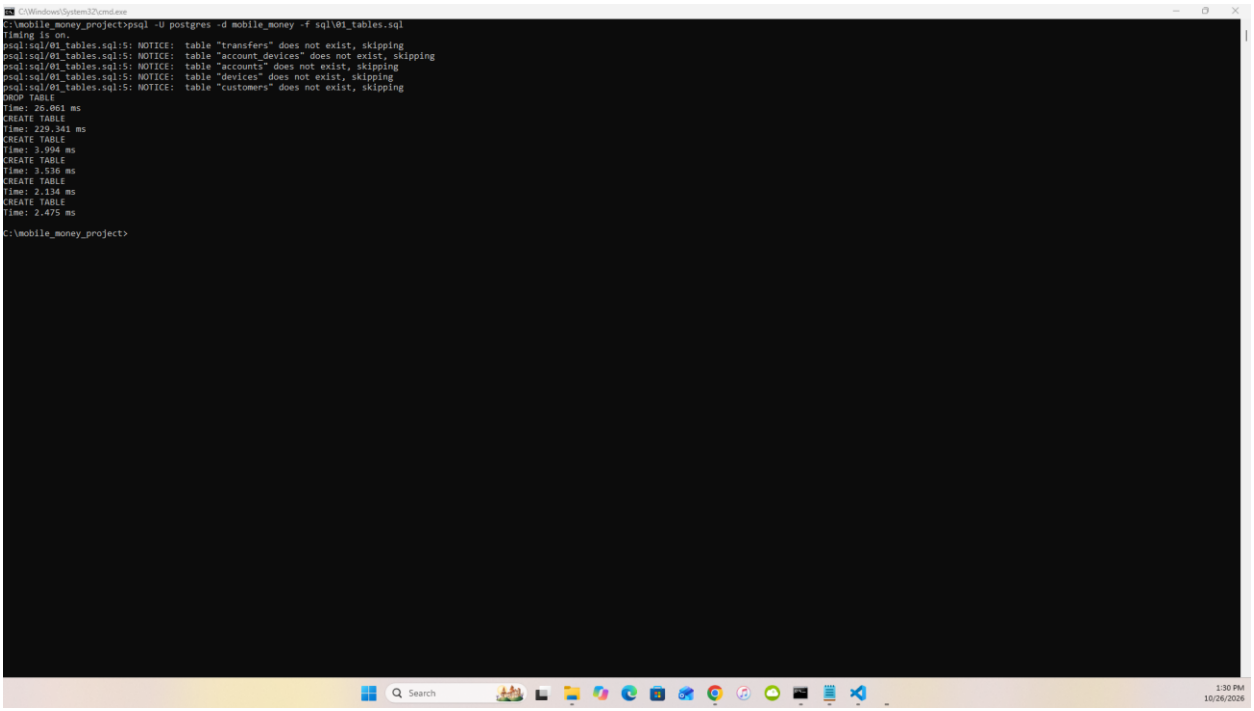
2.1 Schema



PostgreSQL schema

Table	Field	Meaning and unit
customers	customer_id, region	customer key; one of five provinces/Kigali
accounts	account_id, customer_id, role	wallet key; owning customer (FK); person / merchant / agent
devices	device_id, access_type	device key; handset or public_terminal
account_devices	account_id, device_id	an observed (wallet, device) pair – composite PK, both FKs
	first_day, last_day	inclusive day numbers 0–29 (0 = 1 Sep 2026 CAT)
	observations	number of sightings in that interval (> 0)
transfers	transfer_id	event key (BIGINT)
	sender_id, receiver_id	paying and receiving wallets (two FKs to accounts)
	occurred_at	Unix seconds (BIGINT); shown in Africa/Kigali
	amount_rwf	whole Rwandan francs (> 0)

All 17 columns are NOT NULL. Constraints: 5 primary keys, 5 foreign keys, amount_rwf > 0, observations > 0, 0 <= first_day <= last_day <= 29, plus role/access_type value checks. PostgreSQL 18 also records every NOT NULL column as a named, validated constraint (17 more), so the catalogue lists 32 constraints in total, all convalidated = true.



2.1

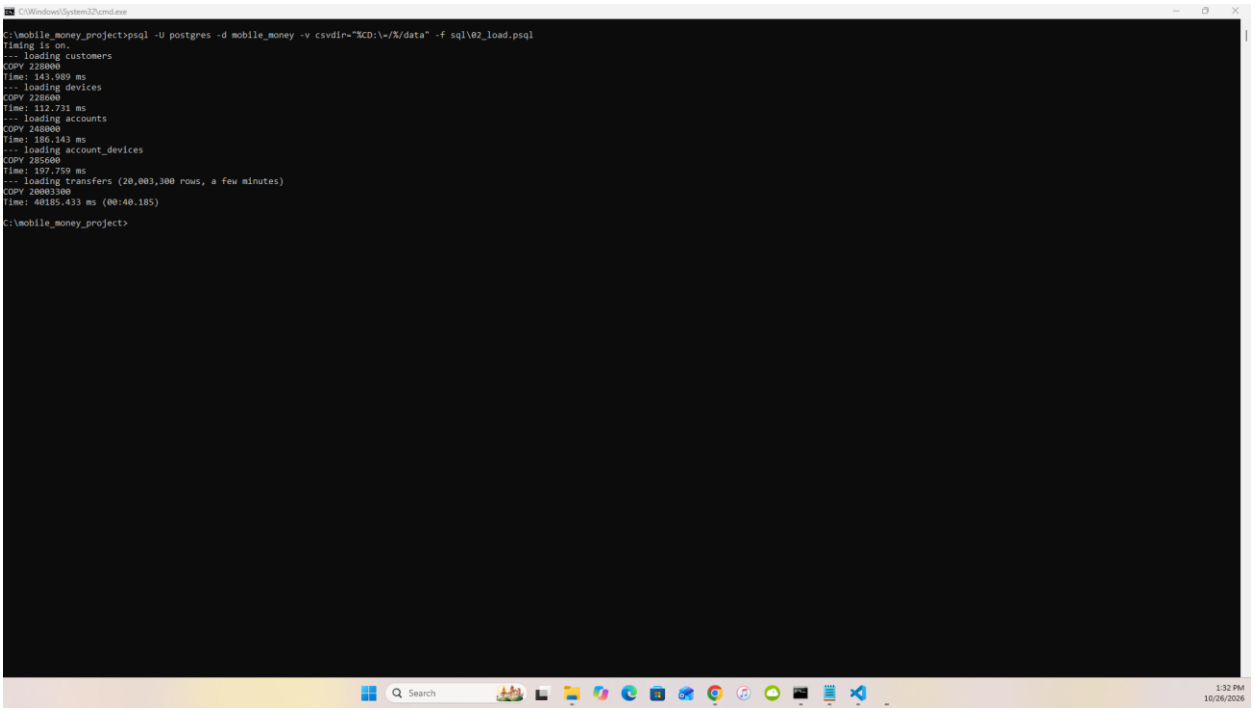
2.2 Load and checks

Phase	Seconds
\copy load of five files	**40.8 s** – customers 0.14 s, devices 0.11 s, accounts 0.19 s, account_devices 0.20 s, transfers **40.19 s** (≈498,000 rows/s)
PK/FK/CHECK + indexes + VACUUM ANALYZE	**325.7 s** – 5 primary keys 75.9 s (transfers PK 73.5 s), 5 foreign keys 17.2 s, CHECKs 2.3 s, secondary indexes 206.1 s (the two covering transfer indexes 84.1 s and 117.4 s), VACUUM ANALYZE 24.3 s

Building keys and indexes took **8× longer than loading the data** (325.7 s vs 40.8 s); 62 % of it went on the two covering B-tree indexes over 20 M transfers. Loading first and indexing afterwards lets each index be built in one sorted pass instead of being updated row by row. Row counts and the transfer audit match capture.py exactly; roles are 240,000 person, 6,000 merchant, 2,000 agent; every constraint is convalidated = true.

Size (1 GB = 10^9 B)	Data	Indexes	Total
all five tables	1.250	2.468	3.718
whole database on disk			3.726

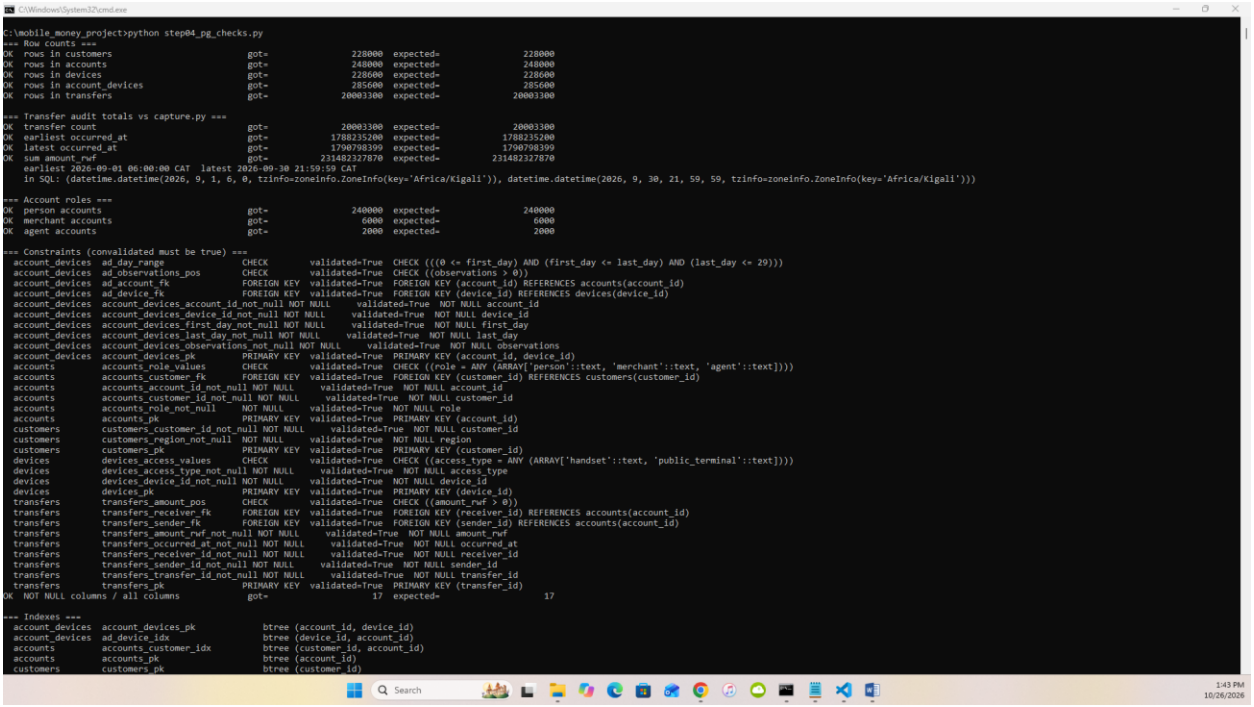
Indexes take **2.47 GB, almost twice the 1.25 GB of table data**. Nearly all of it is the two covering B-tree indexes on transfers (sender, time and receiver, time, each with INCLUDE columns): the price of index-only scans in both transfer directions, which Q1, Q5, Q6 and Q7 rely on. PostgreSQL needs 3.73 GB on disk against 0.80 GB of CSV and 2.05 GB for the Neo4j store.



```
C:\mobile_money_project>psql -U postgres -d mobile_money -v csvdir="%CD:\=/%/data" -f sql\02_load.psql
Timing is on.
--- loading customers
COPY 228000
Time: 143.989 ms
--- loading devices
COPY 228600
Time: 112.731 ms
--- loading accounts
COPY 248000
Time: 186.143 ms
--- loading account_devices
COPY 285600
Time: 197.759 ms
--- loading transfers (20,003,300 rows, a few minutes)
COPY 20003300
Time: 40185.433 ms (00:40.185)

C:\mobile_money_project>
```

2.2a load timings



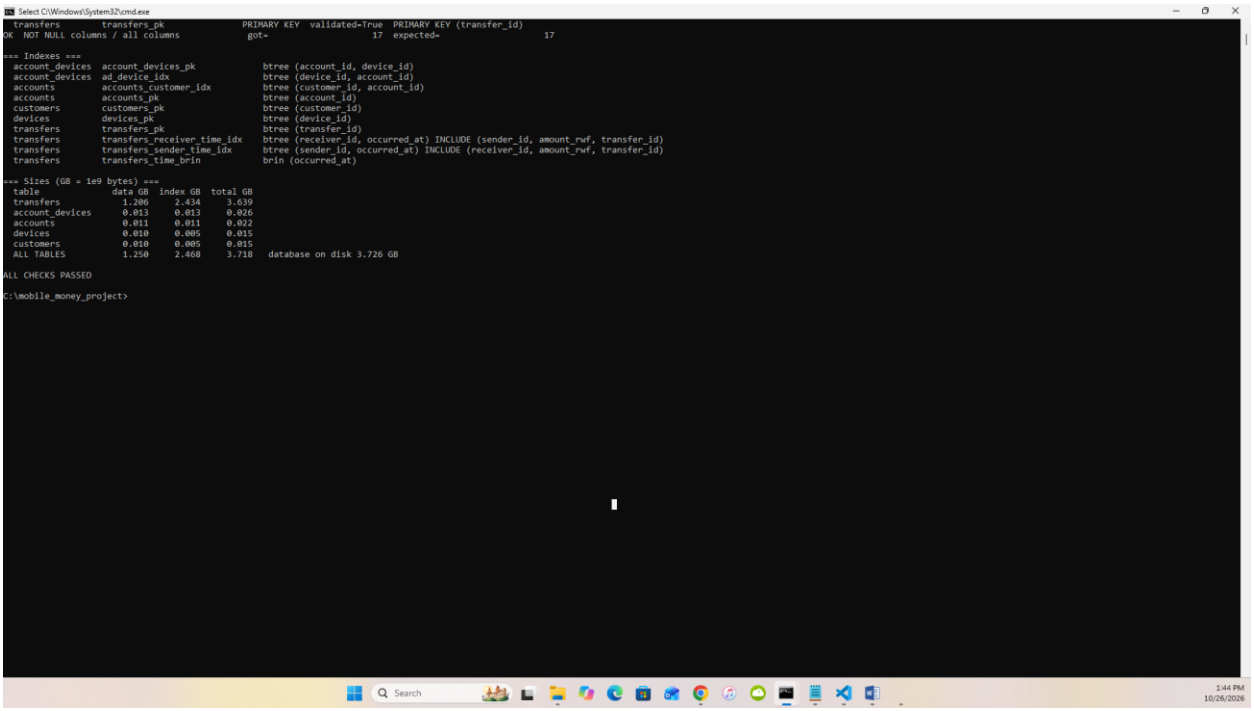
```
C:\mobile_money_project>python step04_pg_checks.py
=== Row Counts ===
OK  rows in customers                        got=              228000  expected=              228000
OK  rows in accounts                         got=              248000  expected=              248000
OK  rows in devices                          got=              228600  expected=              228600
OK  rows in account_devices                  got=              285600  expected=              285600
OK  rows in transfers                        got=            20003300  expected=            20003300

=== Transfer audit totals vs capture.py ===
OK  transfer count                           got=            20003300  expected=            20003300
OK  earliest occurred_at                     got=          1788235200  expected=          1788235200
OK  latest occurred_at                       got=          1790798399  expected=          1790798399
OK  sum amount_rwf                           got=        231482327870  expected=        231482327870
    earliest 2026-09-01 06:00:00 CAT  latest 2026-09-30 21:59:59 CAT
    in SQL: (datetime.datetime(2026, 9, 1, 6, 0, tzinfo=zoneinfo.ZoneInfo(key='Africa/Kigali')), datetime.datetime(2026, 9, 30, 21, 59, 59, tzinfo=zoneinfo.ZoneInfo(key='Africa/Kigali')))

=== Account roles ===
OK  person accounts                          got=              240000  expected=              240000
OK  merchant accounts                        got=                6000  expected=                6000
OK  agent accounts                           got=                2000  expected=                2000

=== Constraints (convalidated must be true) ===
  account_devices  ad_day_range              CHECK        validated=True  CHECK (((0 <= first_day) AND (first_day <= last_day) AND (last_day <= 29)))
  account_devices  ad_observations_pos       CHECK        validated=True  CHECK ((observations > 0))
  account_devices  ad_account_fk             FOREIGN KEY  validated=True  FOREIGN KEY (account_id) REFERENCES accounts(account_id)
  account_devices  ad_device_fk              FOREIGN KEY  validated=True  FOREIGN KEY (device_id) REFERENCES devices(device_id)
  account_devices  account_devices_account_id_not_null NOT NULL     validated=True  NOT NULL account_id
  account_devices  account_devices_device_id_not_null NOT NULL     validated=True  NOT NULL device_id
  account_devices  account_devices_first_day_not_null NOT NULL     validated=True  NOT NULL first_day
  account_devices  account_devices_last_day_not_null NOT NULL     validated=True  NOT NULL last_day
  account_devices  account_devices_observations_not_null NOT NULL     validated=True  NOT NULL observations
  account_devices  account_devices_pk        PRIMARY KEY  validated=True  PRIMARY KEY (account_id, device_id)
  accounts         accounts_role_values      CHECK        validated=True  CHECK ((role = ANY (ARRAY['person'::text, 'merchant'::text, 'agent'::text])))
  accounts         accounts_customer_fk      FOREIGN KEY  validated=True  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
  accounts         accounts_account_id_not_null NOT NULL     validated=True  NOT NULL account_id
  accounts         accounts_customer_id_not_null NOT NULL     validated=True  NOT NULL customer_id
  accounts         accounts_role_not_null    NOT NULL     validated=True  NOT NULL role
  accounts         accounts_pk               PRIMARY KEY  validated=True  PRIMARY KEY (account_id)
  customers        customers_customer_id_not_null NOT NULL     validated=True  NOT NULL customer_id
  customers        customers_region_not_null  NOT NULL     validated=True  NOT NULL region
  customers        customers_pk              PRIMARY KEY  validated=True  PRIMARY KEY (customer_id)
  devices          devices_access_values     CHECK        validated=True  CHECK ((access_type = ANY (ARRAY['handset'::text, 'public_terminal'::text])))
  devices          devices_access_type_not_null NOT NULL     validated=True  NOT NULL access_type
  devices          devices_device_id_not_null NOT NULL     validated=True  NOT NULL device_id
  devices          devices_pk                PRIMARY KEY  validated=True  PRIMARY KEY (device_id)
  transfers        transfers_amount_pos      CHECK        validated=True  CHECK ((amount_rwf > 0))
  transfers        transfers_receiver_fk     FOREIGN KEY  validated=True  FOREIGN KEY (receiver_id) REFERENCES accounts(account_id)
  transfers        transfers_sender_fk       FOREIGN KEY  validated=True  FOREIGN KEY (sender_id) REFERENCES accounts(account_id)
  transfers        transfers_amount_rwf_not_null NOT NULL     validated=True  NOT NULL amount_rwf
  transfers        transfers_occurred_at_not_null NOT NULL     validated=True  NOT NULL occurred_at
  transfers        transfers_receiver_id_not_null NOT NULL     validated=True  NOT NULL receiver_id
  transfers        transfers_sender_id_not_null NOT NULL     validated=True  NOT NULL sender_id
  transfers        transfers_transfer_id_not_null NOT NULL     validated=True  NOT NULL transfer_id
  transfers        transfers_pk              PRIMARY KEY  validated=True  PRIMARY KEY (transfer_id)
OK  NOT NULL columns / all columns           got=                17  expected=                17

=== Indexes ===
  account_devices  account_devices_pk            btree (account_id, device_id)
  account_devices  ad_device_idx                 btree (device_id, account_id)
  accounts         accounts_customer_idx         btree (customer_id, account_id)
  accounts         accounts_pk                   btree (account_id)
  customers        customers_pk                  btree (customer_id)
```

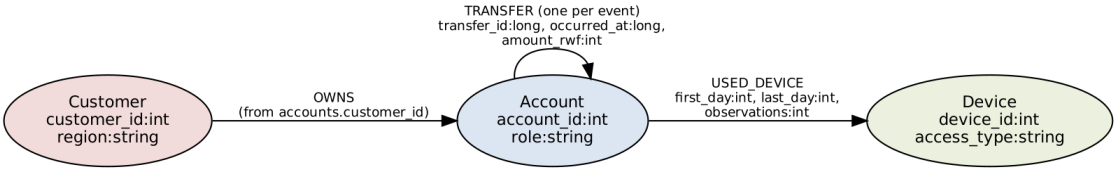


2.2b step04 checks, constraints, indexes, sizes

---

3 Neo4j

3.1 Graph mapping



Graph schema

Source field	Graph
customers.customer_id, region	(:Customer {customer_id:int, region})
accounts.account_id, role	(:Account {account_id:int, role})
accounts.customer_id	(:Customer)-[:OWNS]->(:Account) (derived owns.csv)
devices.device_id, access_type	(:Device {device_id:int, access_type})
account_devices.account_id / device_id	start / end of [:USED_DEVICE]
account_devices.first_day, last_day, observations	properties of USED_DEVICE
transfers.sender_id / receiver_id	start / end of [:TRANSFER]
transfers.transfer_id, occurred_at, amount_rwf	properties of **one TRANSFER per row** (repeated payments kept)

3.2 Import

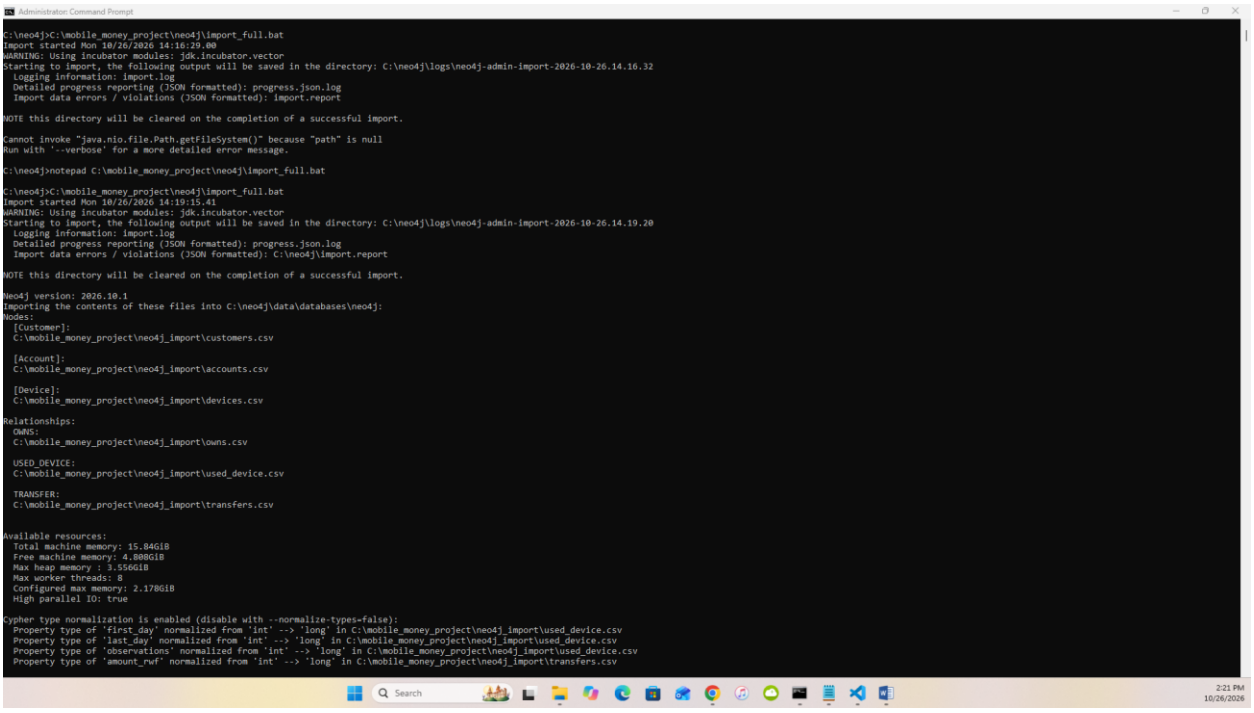
Typed headers (temporary copies; source CSVs unchanged, verified by MD5): customer_id:ID(Customer),region:string · account_id:ID(Account),:IGNORE,role:string · device_id:ID(Device),access_type:string · owns.csv :START_ID(Customer),:END_ID(Account) · used_device.csv :START_ID(Account),:END_ID(Device),first_day:int,last_day:int,observations:int · transfers.csv transfer_id:long,:START_ID(Account),:END_ID(Account),occurred_at:long,amount_rwf:int.

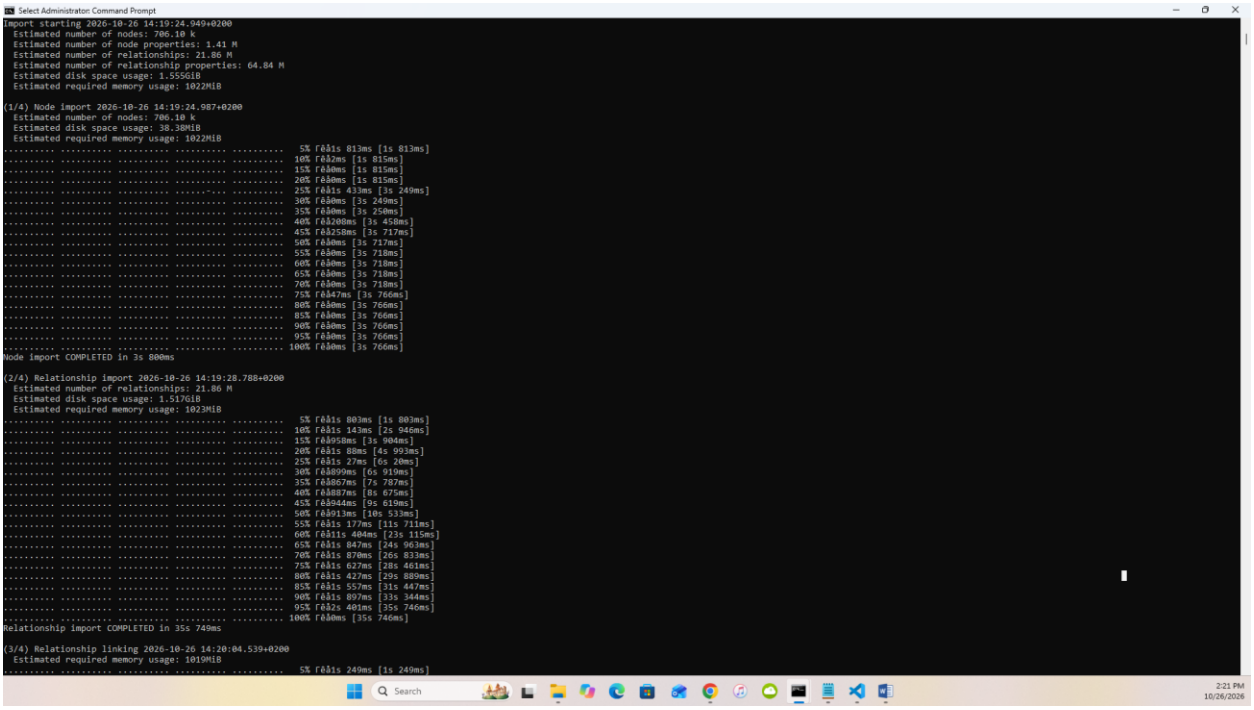
neo4j-admin database import full neo4j --overwrite-destination=true --id-type=integer --nodes=Customer=customers.csv --nodes=Account=accounts.csv --nodes=Device=devices.csv --relationships=OWNS=owns.csv --relationships=USED_DEVICE=used_device.csv --relationships=TRANSFER=transfers.csv

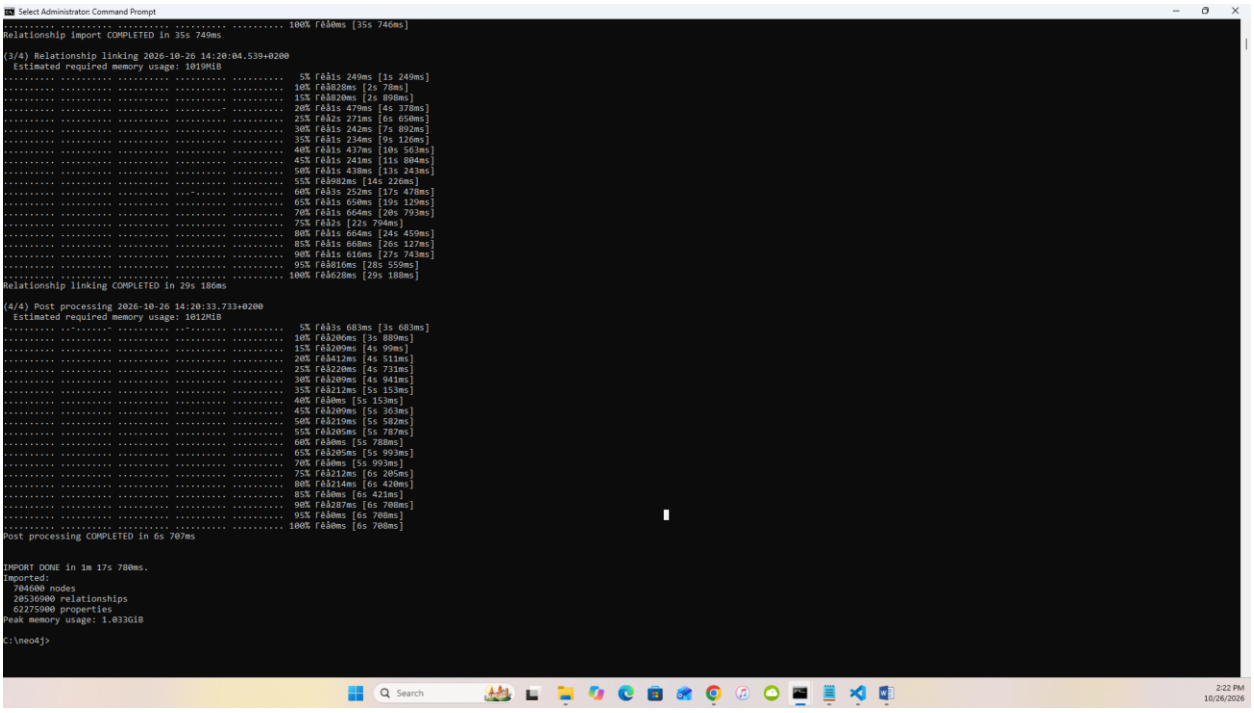
Item	Value
Import seconds	**77.8 s** (nodes 3.8 s, relationships 35.7 s, linking 29.2 s, post-processing 6.7 s; peak memory 1.03 GB)
Constraint + index setup (until ONLINE)	**103.8 s**
Store GB / transaction-log GB	**2.049 GB** / ≈0 GB (an offline import writes store files directly, not through the transaction log)

Neo4j 2026.10 applied *Cypher type normalisation*: the :int columns (first_day, last_day, observations, amount_rwf) were stored as 64-bit long, Cypher's only integer type. Values and comparisons are unchanged. Imported: 704,600 nodes, 20,536,900 relationships, 62,275,900 properties.

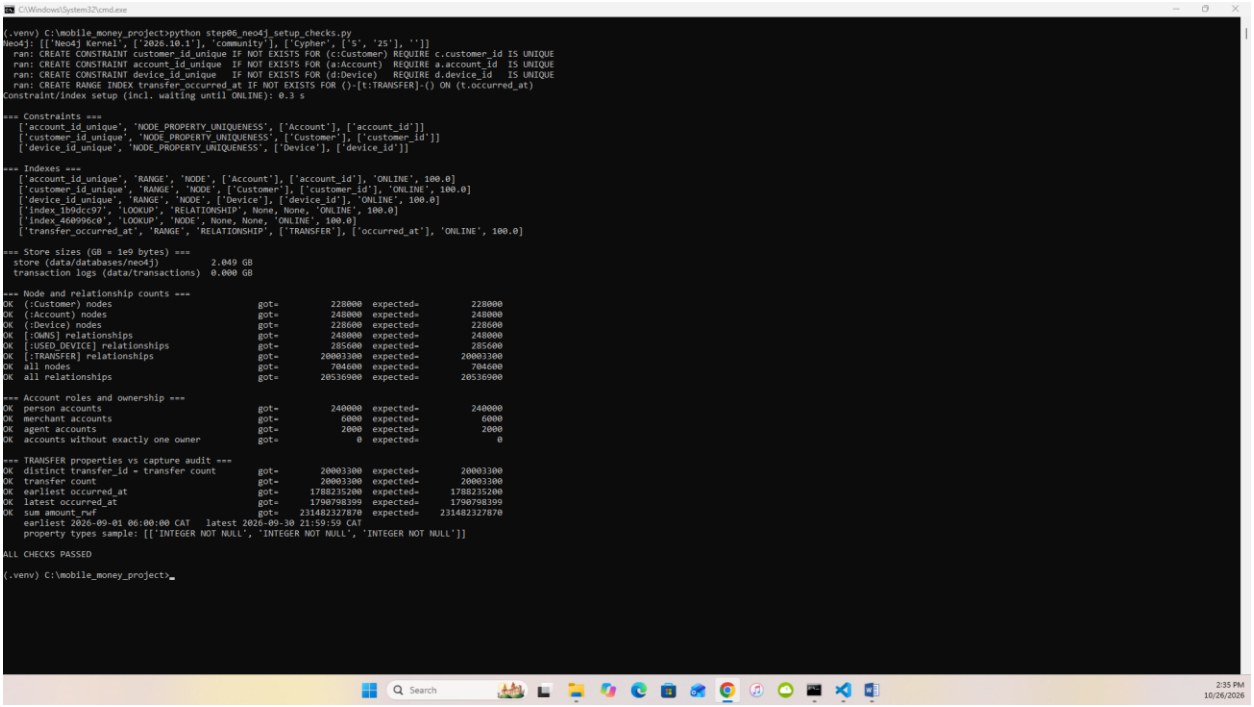
Checks: Customer 228,000 · Account 248,000 · Device 228,600 · OWNS 248,000 · USED_DEVICE 285,600 · TRANSFER 20,003,300 · 704,600 nodes · 20,536,900 relationships · roles as in SQL · 0 accounts without exactly one owner · distinct transfer_id = 20,003,300 · audit totals equal.

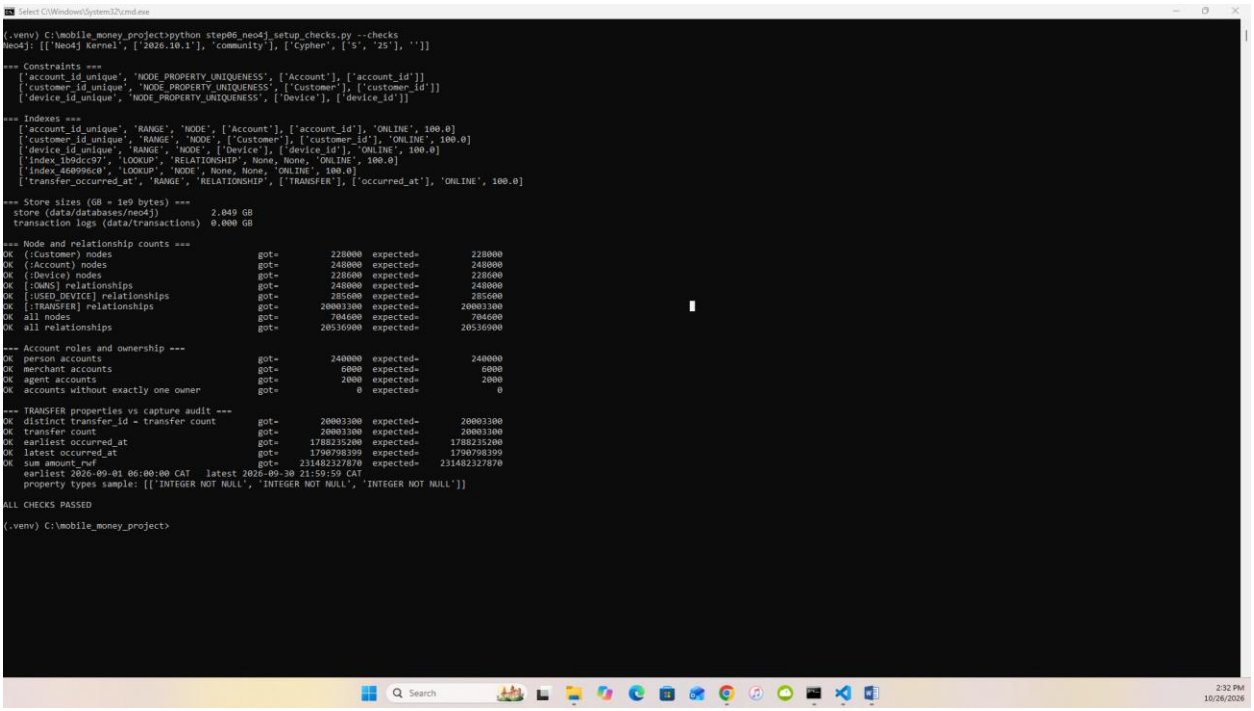

Administrator: Command Prompt
C:\neo4j>C:\mobile_money_project\neo4j\import_full.bat
Import started Mon 10/26/2026 14:16:29.00
WARNING: Using incubator modules: jdk.incubator.vector
Starting to import, the following output will be saved in the directory: C:\neo4j\logs\neo4j-admin-import-2026-10-26.14.16.32
Logging information: import.log
Detailed progress reporting (JSON formatted): progress.json.log
Import data errors / violations (JSON formatted): import.report
NOTE this directory will be cleared on the completion of a successful import.
Cannot invoke "java.nio.file.Path.getFileSystem()" because "path" is null
Run with '--verbose' for a more detailed error message.
C:\neo4j>notepad C:\mobile_money_project\neo4j\import_full.bat
C:\neo4j>C:\mobile_money_project\neo4j\import_full.bat
Import started Mon 10/26/2026 14:19:15.41
WARNING: Using incubator modules: jdk.incubator.vector
Starting to import, the following output will be saved in the directory: C:\neo4j\logs\neo4j-admin-import-2026-10-26.14.19.20
Logging information: import.log
Detailed progress reporting (JSON formatted): progress.json.log
Import data errors / violations (JSON formatted): C:\neo4j\import.report
NOTE this directory will be cleared on the completion of a successful import.
Neo4j version: 2026.10.1
Importing the contents of these files into C:\neo4j\data\databases\neo4j:
Nodes:
[Customer]:
C:\mobile_money_project\neo4j_import\customers.csv
[Account]:
C:\mobile_money_project\neo4j_import\accounts.csv
[Device]:
C:\mobile_money_project\neo4j_import\devices.csv
Relationships:
OWNS:
C:\mobile_money_project\neo4j_import\owns.csv
USED_DEVICE:
C:\mobile_money_project\neo4j_import\used_device.csv
TRANSFER:
C:\mobile_money_project\neo4j_import\transfers.csv
Available resources:
Total machine memory: 15.84GiB
Free machine memory: 4.808GiB
Max heap memory : 3.556GiB
Max worker threads: 8
Configured max memory: 2.178GiB
High parallel IO: true
Cypher type normalization is enabled (disable with --normalize-types=false):
Property type of 'first_day' normalized from 'int' --> 'long' in C:\mobile_money_project\neo4j_import\used_device.csv
Property type of 'last_day' normalized from 'int' --> 'long' in C:\mobile_money_project\neo4j_import\used_device.csv
Property type of 'observations' normalized from 'int' --> 'long' in C:\mobile_money_project\neo4j_import\used_device.csv
Property type of 'amount_rwf' normalized from 'int' --> 'long' in C:\mobile_money_project\neo4j_import\transfers.csv
Search
2:21 PM
10/26/2026


Select Administrator: Command Prompt
Import starting 2026-10-26 14:19:24.949+0200
Estimated number of nodes: 706.10 k
Estimated number of node properties: 1.41 M
Estimated number of relationships: 21.86 M
Estimated number of relationship properties: 64.84 M
Estimated disk space usage: 1.555GiB
Estimated required memory usage: 1022MiB
(1/4) Node import 2026-10-26 14:19:24.987+0200
Estimated number of nodes: 706.10 k
Estimated disk space usage: 38.38MiB
Estimated required memory usage: 1022MiB
5% 1s 813ms [1s 813ms]
10% 2ms [1s 815ms]
15% 0ms [1s 815ms]
20% 0ms [1s 815ms]
25% 1s 433ms [3s 249ms]
30% 0ms [3s 249ms]
35% 0ms [3s 250ms]
40% 208ms [3s 458ms]
45% 258ms [3s 717ms]
50% 0ms [3s 717ms]
55% 0ms [3s 718ms]
60% 0ms [3s 718ms]
65% 0ms [3s 718ms]
70% 0ms [3s 718ms]
75% 47ms [3s 766ms]
80% 0ms [3s 766ms]
85% 0ms [3s 766ms]
90% 0ms [3s 766ms]
95% 0ms [3s 766ms]
100% 0ms [3s 766ms]
Node import COMPLETED in 3s 800ms
(2/4) Relationship import 2026-10-26 14:19:28.788+0200
Estimated number of relationships: 21.86 M
Estimated disk space usage: 1.517GiB
Estimated required memory usage: 1023MiB
5% 1s 803ms [1s 803ms]
10% 1s 143ms [2s 946ms]
15% 958ms [3s 904ms]
20% 1s 88ms [4s 993ms]
25% 1s 27ms [6s 20ms]
30% 899ms [6s 919ms]
35% 867ms [7s 787ms]
40% 887ms [8s 675ms]
45% 944ms [9s 619ms]
50% 913ms [10s 533ms]
55% 1s 177ms [11s 711ms]
60% 1s 404ms [23s 115ms]
65% 1s 847ms [24s 963ms]
70% 1s 870ms [26s 833ms]
75% 1s 627ms [28s 461ms]
80% 1s 427ms [29s 889ms]
85% 1s 557ms [31s 447ms]
90% 1s 897ms [33s 344ms]
95% 2s 401ms [35s 746ms]
100% 0ms [35s 746ms]
Relationship import COMPLETED in 35s 749ms
(3/4) Relationship linking 2026-10-26 14:20:04.539+0200
Estimated required memory usage: 1019MiB
5% 1s 249ms [1s 249ms]
Search
2:21 PM
10/26/2026

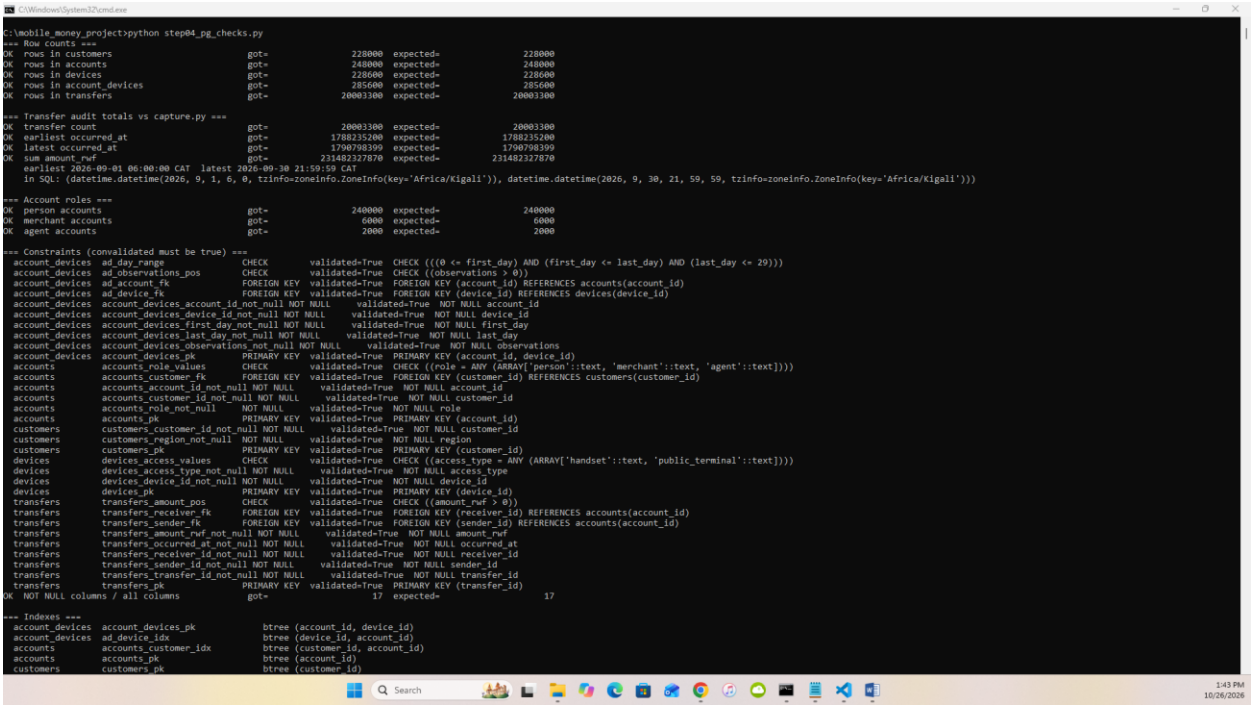

Select Administrator: Command Prompt
Relationship import COMPLETED in 35s 749ms
(3/4) Relationship linking 2026-10-26 14:20:04.539+0200
Estimated required memory usage: 1019MiB
Relationship linking COMPLETED in 29s 186ms
(4/4) Post processing 2026-10-26 14:20:33.733+0200
Estimated required memory usage: 1012MiB
Post processing COMPLETED in 6s 707ms
IMPORT DONE in 1m 17s 780ms.
Imported:
704600 nodes
20536900 relationships
62275900 properties
Peak memory usage: 1.033GiB
C:\neo4j>
Search
2:22 PM
10/26/2026

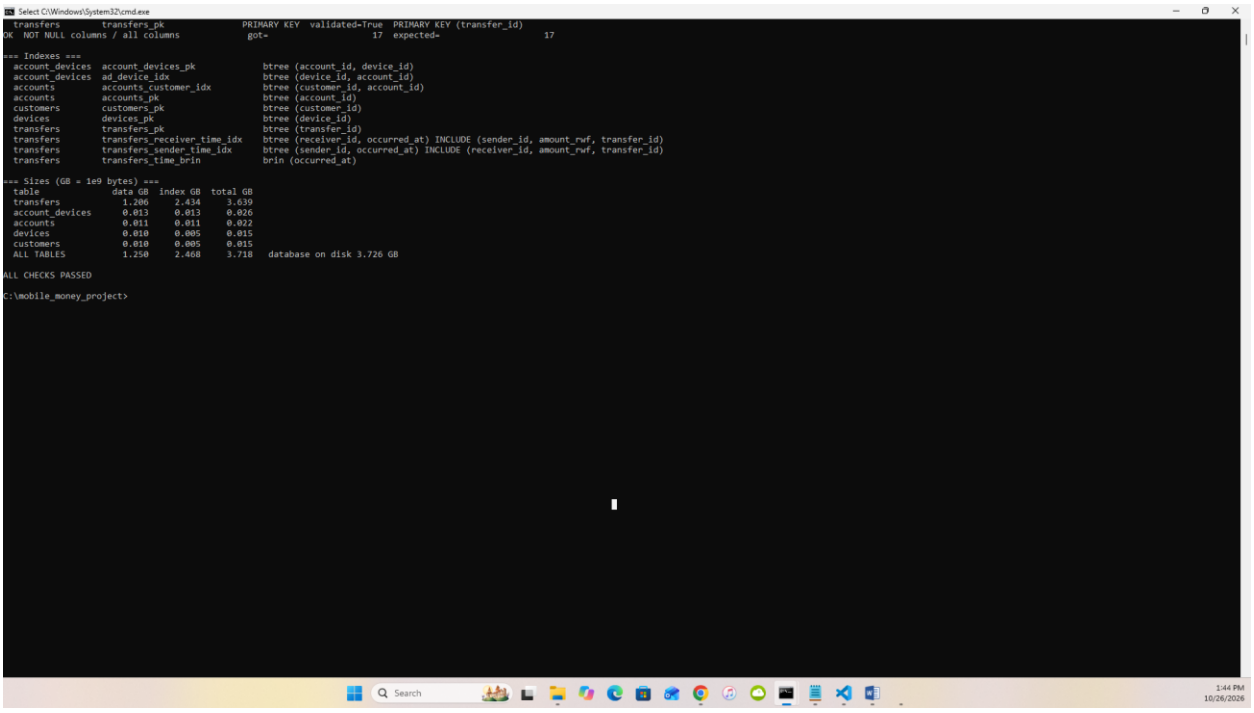
3.2a import


C:\Windows\System32\cmd.exe
(.venv) C:\mobile_money_project>python step06_neo4j_setup_checks.py
ran: CREATE CONSTRAINT customer_id_unique IF NOT EXISTS FOR (c:Customer) REQUIRE c.customer_id IS UNIQUE
ran: CREATE CONSTRAINT account_id_unique IF NOT EXISTS FOR (a:Account) REQUIRE a.account_id IS UNIQUE
ran: CREATE CONSTRAINT device_id_unique IF NOT EXISTS FOR (d:Device) REQUIRE d.device_id IS UNIQUE
ran: CREATE RANGE INDEX transfer_occurred_at IF NOT EXISTS FOR ()-[t:TRANSFER]-() ON (t.occurred_at)
Constraint/index setup (incl. waiting until ONLINE): 0.3 s
=== Constraints ===
=== Indexes ===
=== Store sizes (GB = 1e9 bytes) ===
store (data/databases/neo4j) 2.049 GB
transaction logs (data/transactions) 0.000 GB
=== Node and relationship counts ===
OK (:Customer) nodes got= 228000 expected= 228000
OK (:Account) nodes got= 248000 expected= 248000
OK (:Device) nodes got= 228600 expected= 228600
OK [:OWNS] relationships got= 248000 expected= 248000
OK [:USED_DEVICE] relationships got= 285600 expected= 285600
OK [:TRANSFER] relationships got= 20003300 expected= 20003300
OK all nodes got= 704600 expected= 704600
OK all relationships got= 20536900 expected= 20536900
=== Account roles and ownership ===
OK person accounts got= 240000 expected= 240000
OK merchant accounts got= 6000 expected= 6000
OK agent accounts got= 2000 expected= 2000
OK accounts without exactly one owner got= 0 expected= 0
=== TRANSFER properties vs capture audit ===
OK distinct transfer_id = transfer count got= 20003300 expected= 20003300
OK transfer count got= 20003300 expected= 20003300
OK earliest occurred_at got= 1788235200 expected= 1788235200
OK latest occurred_at got= 1790798399 expected= 1790798399
OK sum amount_rwf got= 231482327870 expected= 231482327870
earliest 2026-09-01 06:00:00 CAT latest 2026-09-30 21:59:59 CAT
property types sample: [['INTEGER NOT NULL', 'INTEGER NOT NULL', 'INTEGER NOT NULL']]
ALL CHECKS PASSED
(.venv) C:\mobile_money_project>
Search
2:35 PM
10/26/2026

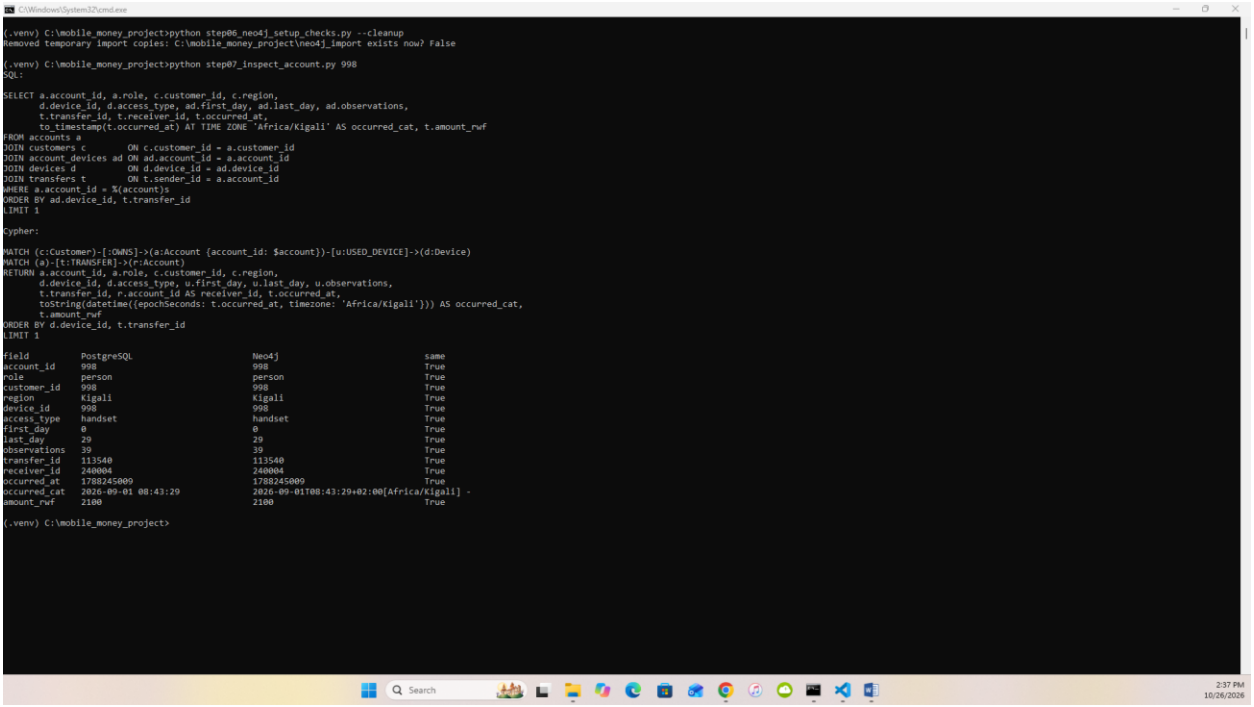


3.2b constraints/indexes





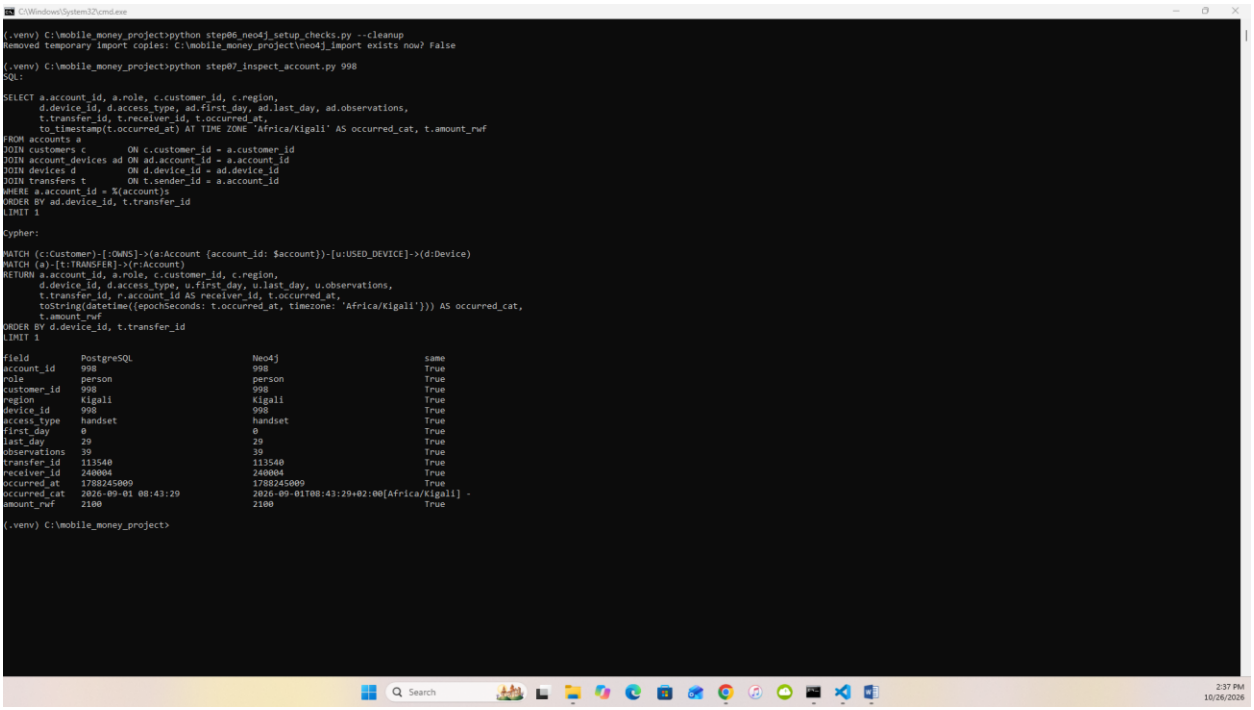
3.2c checks



3.2d cleanup – temporary import copies removed

3.3 One account in both models

Account 998 (person) → owner Customer 998 (Kigali) → Device 998 (handset, days 0–29, 39 observations) → first outgoing transfer 113540 to merchant 240004 on 2026-09-01 08:43:29 CAT, 2,100 RWF. Same record from JOIN accounts/customers/account_devices/devices/transfers and from MATCH (c)-[:OWNS]->(a)-[:USED_DEVICE]->(d), (a)-[:TRANSFER]->(r).



3.3

Foreign keys become relationships. accounts.customer_id becomes an OWNS relationship; the account_devices row (a junction table) becomes a USED_DEVICE relationship carrying its three columns; the two transfer FKs become the start and end nodes of a TRANSFER. A join that SQL resolves at query time through an index is stored in the graph as a direct pointer. **If repeated payments were collapsed into one edge** per sender–receiver pair, each event's transfer_id, exact occurred_at and amount_rwf would be lost (only a count or sum could remain). Q1, Q2, Q5 and Q6 would become impossible: we could no longer test the hour window, strict time order, the 600-second gap, the 1,800-second span or the 90–105 % amount rule.

---

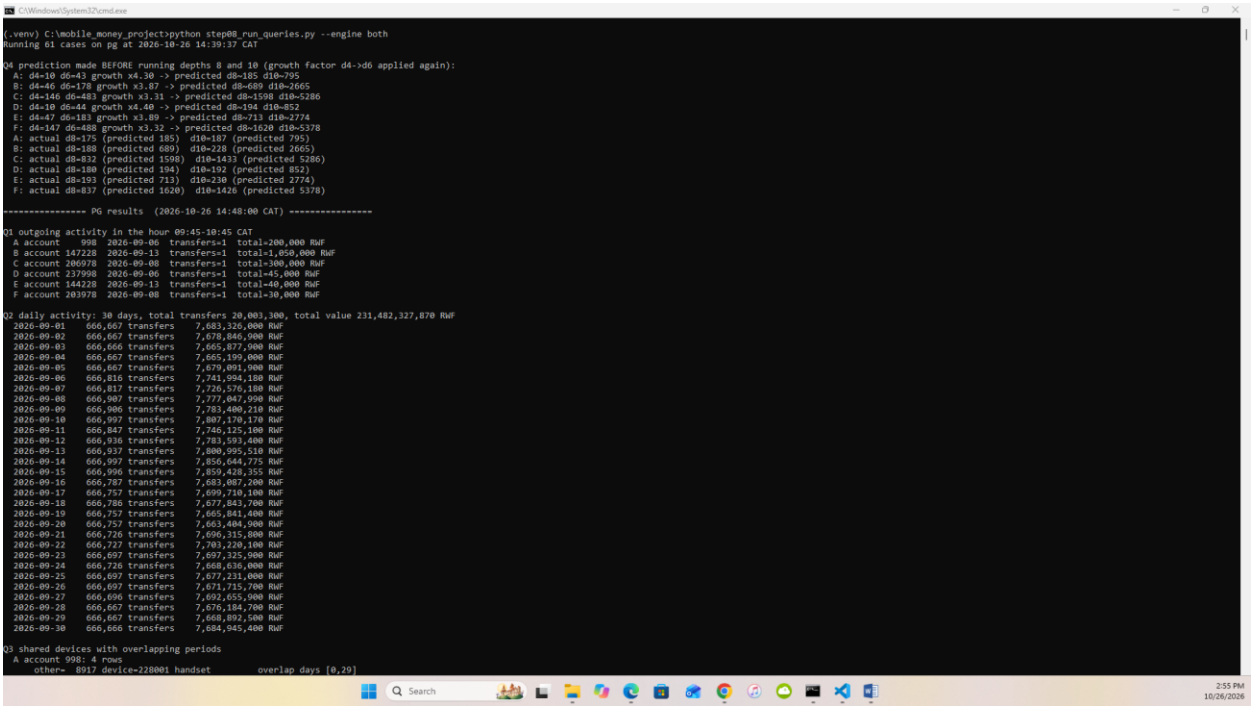
4 Routine business questions

Hour window: 09:45:00 ≤ t < 10:45:00 CAT, converted to epoch seconds in Python (mm_common.hour_bounds). All queries are in queries.py.

4.1 Q1 Account activity

Set	Account	Date	Transfers	Total RWF
A	998	6 Sep	1	200,000
B	147228	13 Sep	1	1,050,000
C	206978	8 Sep	1	300,000
D	237998	6 Sep	1	45,000
E	144228	13 Sep	1	40,000
F	203978	8 Sep	1	30,000

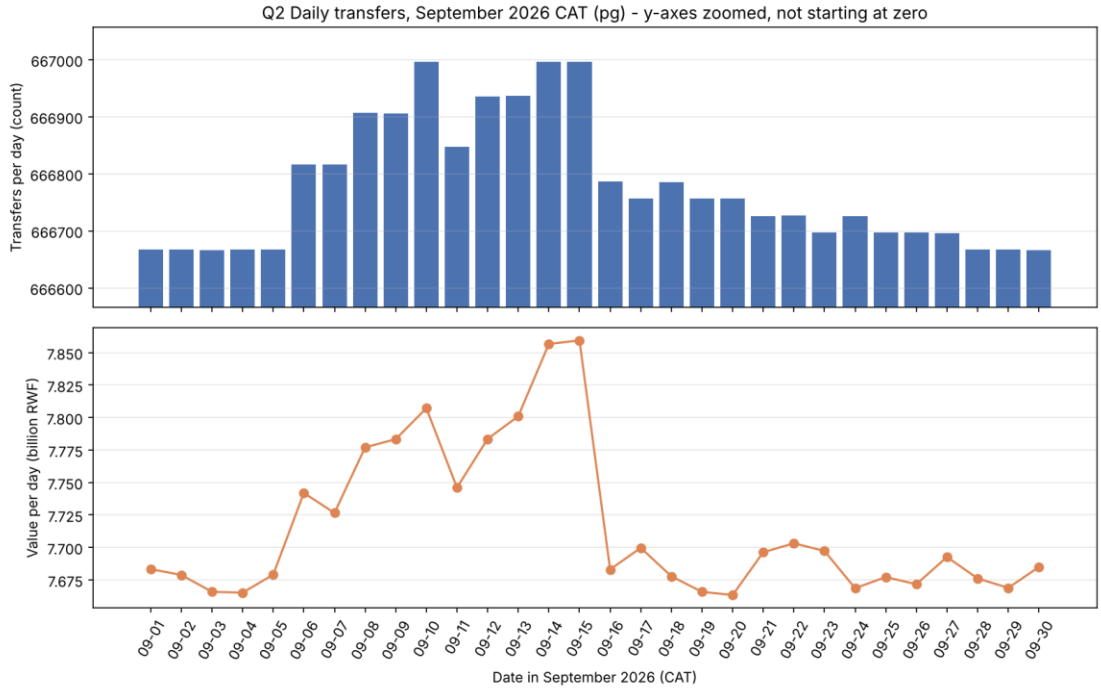
PostgreSQL and Neo4j agree for all six. **Outgoing value is not a balance**: it ignores money received, opening balance, cash-in/cash-out at agents, fees and reversals – the dataset is explicitly not a ledger.



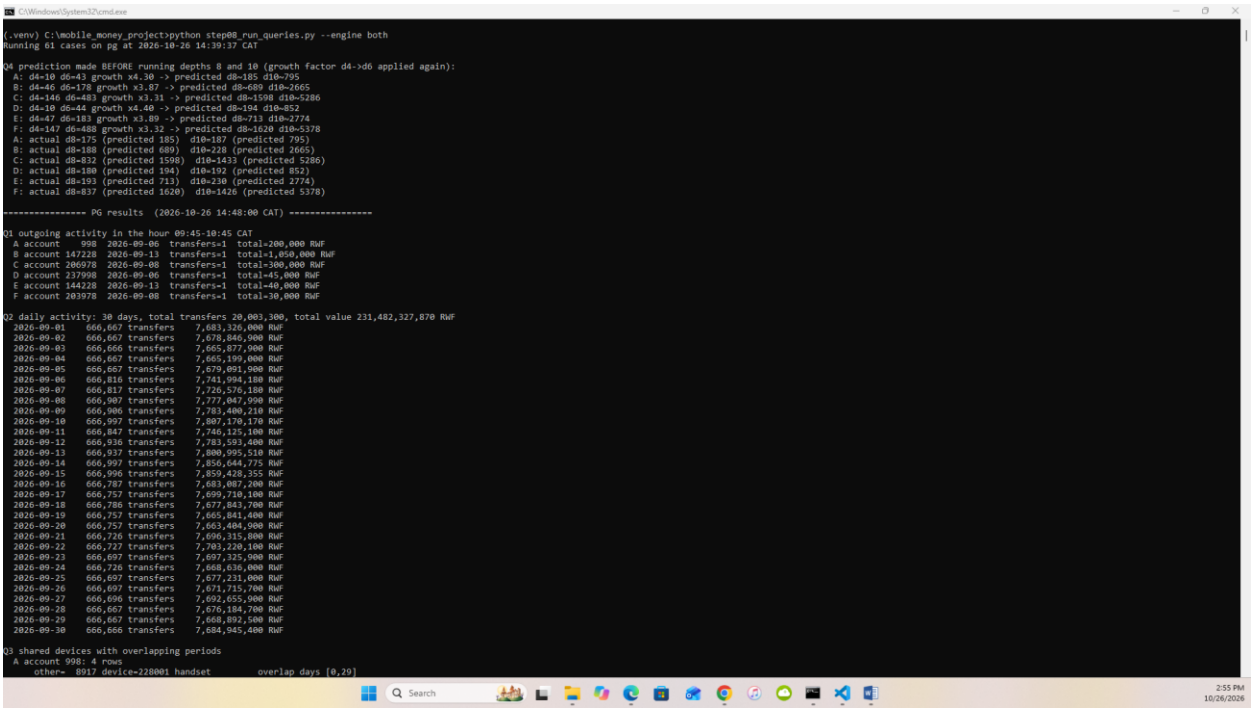
4.1

4.2 Q2 Daily activity

30 rows; sums reconcile to 20,003,300 transfers and 231,482,327,870 RWF.



Pattern. Volume is almost perfectly flat (666,666–666,667 transfers per day from 1–5 Sep and 28–30 Sep), but 6–15 Sep carry an extra 150–330 transfers and up to ≈190 million RWF per day (e.g. 14 Sep: 666,997 transfers, 7.857 bn RWF vs ≈7.67 bn on quiet days), and 16–27 Sep a smaller 30–120. The extra events are the scenario payments: large, fast person-to-person hops (200,000–1,150,000 RWF, the chains found in Q5/Q6) on 6–15 Sep, and small hops repeated every second day afterwards – visible in value far more than in count. All activity falls between 06:00 and 22:00 CAT. **Value is not revenue**: the provider earns fees and commissions, which are not in the data; P2P, merchant and agent flows carry different (or zero) fees, and the same money may be counted several times as it moves wallet to wallet.

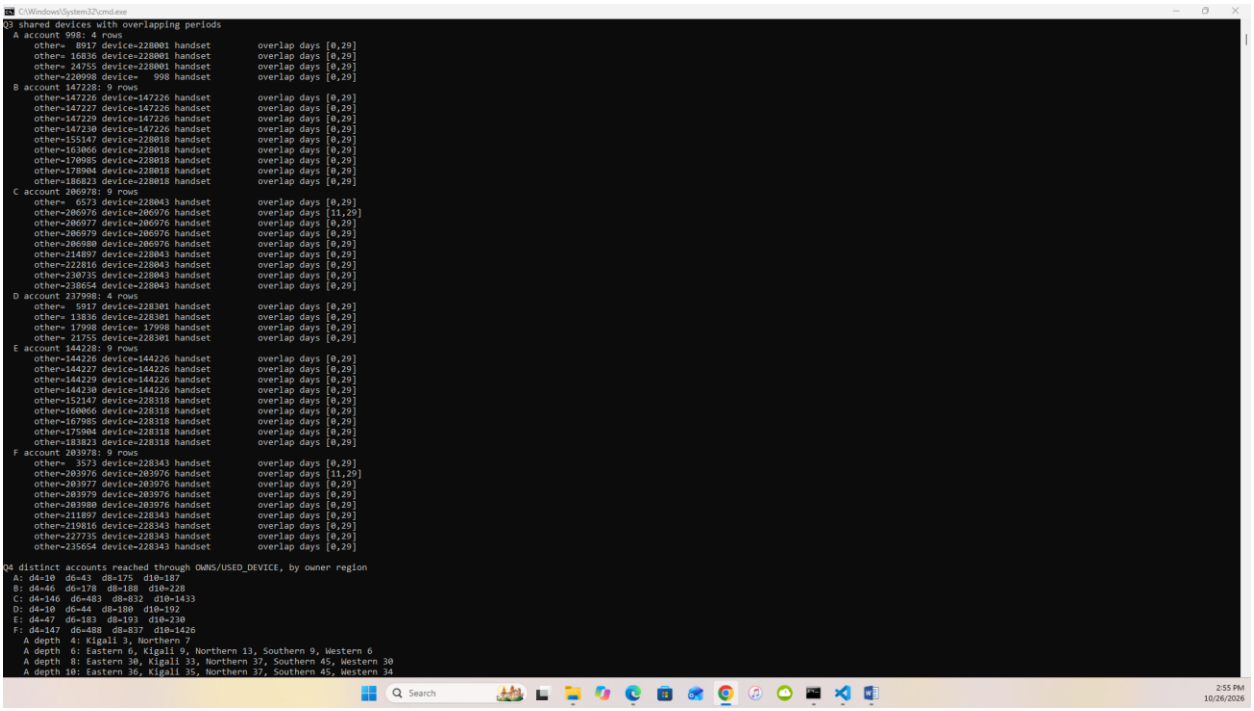


4.2

4.3 Q3 Shared devices

Set	Rows	Devices shared
A 998	4	handset 998 with own second wallet 220998; handset 228001 with 8917, 16836, 24755
B 147228	9	household handset 147226 (4 accounts); handset 228018 with 155147, 163066, 170985, 178904, 186823
C 206978	9	household handset 206976 (4; one overlap only days 11–29); handset 228043 with 5 accounts incl. 6573
D 237998	4	own second wallet 17998 on handset 17998; handset 228301 with 5917, 13836, 21755
E 144228	9	household 144226 (4); handset 228318 with 5 accounts incl. 183823
F 203978	9	household 203976 (4); handset 228343 with 5 accounts incl. 3573

Legitimate sharing. B's handset 147226 is used by wallets of five neighbouring customers – a family sharing one phone; account 206976 appears only from day 11 because that customer replaced their phone. A customer's second wallet naturally appears on their own handset. Public terminals at agents are shared by many unrelated customers by design. **Overlap ≠ simultaneous use.** An interval only says the device was seen with each account at some time between first and last day: [2,7] and [5,11] overlap on [5,7], but one account could have used it on day 5 morning and the other on day 7 evening. The data has no session times.



4.3

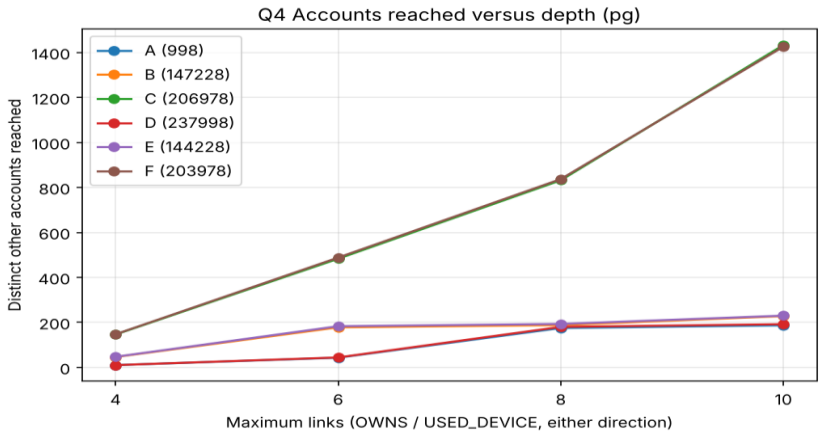
---

5 Connected accounts

5.1 Q4 Ownership and device network

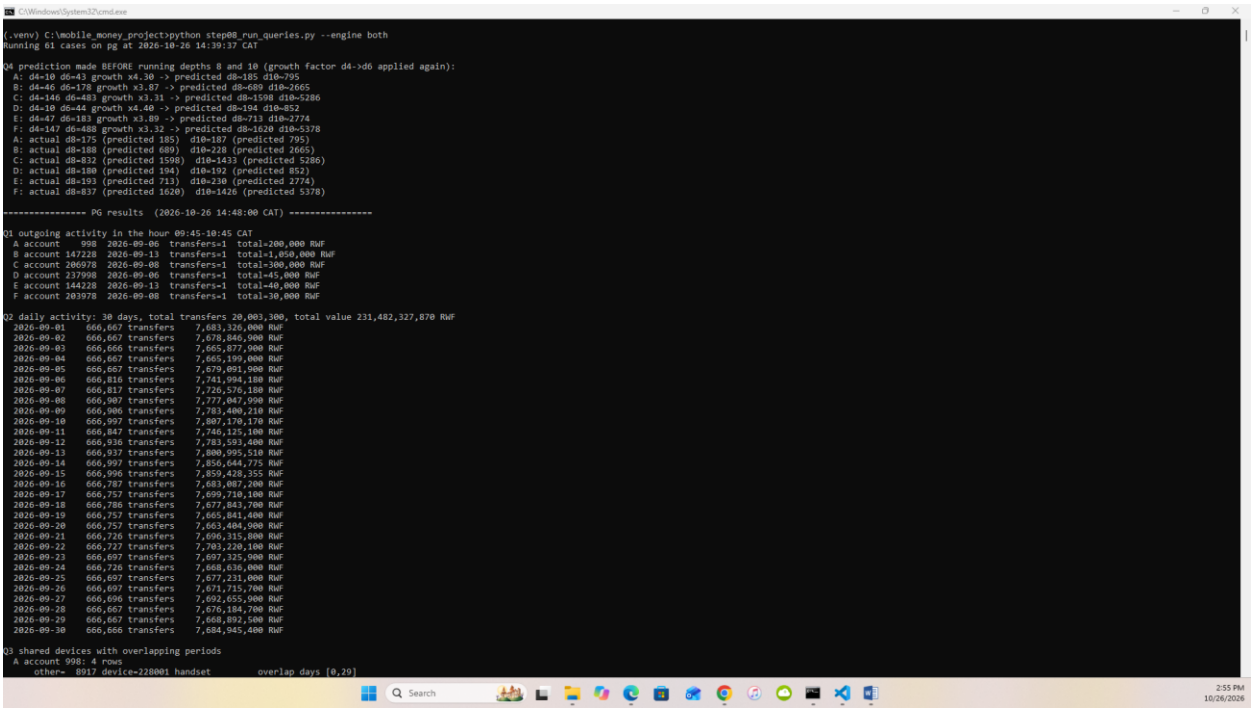
Distinct other accounts reached through OWNS/USED_DEVICE in either direction:

Set	≤4	≤6	≤8	≤10	Predicted 8 / 10 (from 4→6 growth)
A	10	43	175	187	185 / 795
B	46	178	188	228	689 / 2,665
C	146	483	832	1,433	1,598 / 5,286
D	10	44	180	192	194 / 852
E	47	183	193	230	713 / 2,774
F	147	488	837	1,426	1,620 / 5,378



Region breakdowns for all 24 cases are in results/examples_pg.txt; all 24 match in Neo4j.

Supporting path: Account 998 –USED_DEVICE– Device 228001 –USED_DEVICE– Account 24755 (2 links).
Prediction vs result. Constant geometric growth over-predicted depth 10 by 3–12×: the network is made of small household/scenario clusters, and growth stalls once a cluster is exhausted, then jumps only when a path reaches a widely shared device. C and F reach four **public terminals** from depth 3 (each used by dozens of unrelated wallets), so they keep growing to ≈1,430 accounts; A, B, D and E never reach a terminal and level off near 190–230. **Weak associations:** each extra pair of links crosses another device or customer, so at depth 8–10 an account is "connected" only through a chain like *my household phone → a neighbour's second wallet → an agent's public terminal → a stranger*. Such reach reflects shared infrastructure, not a relationship; counts grow while the evidential value per account falls.

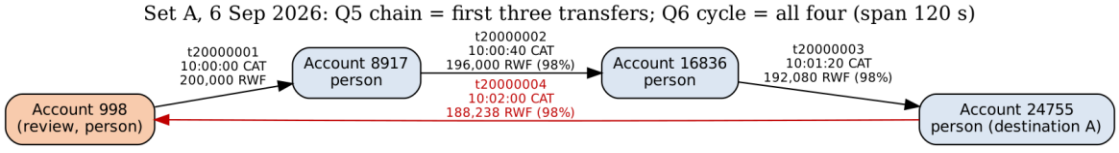


5.1

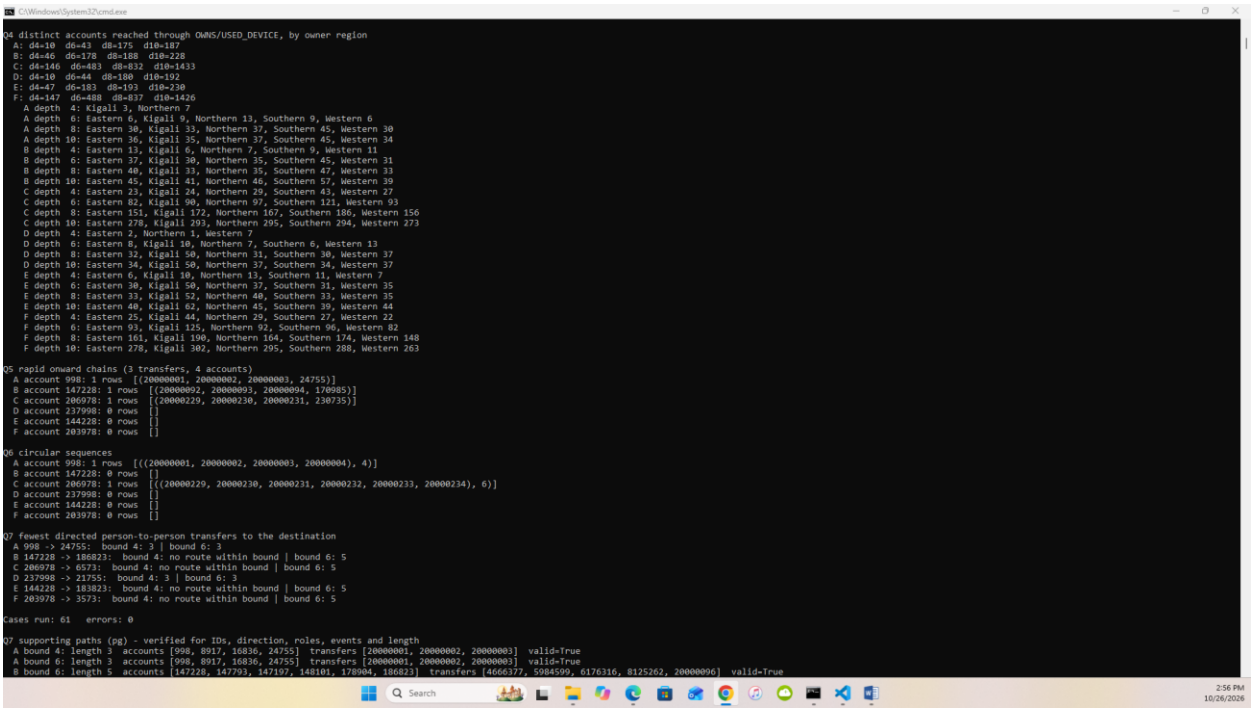
5.2 Q5 Rapid onward payments

Set	Chains	Transfer IDs → final receiver
A	1	20000001, 20000002, 20000003 → 24755
B	1	20000092, 20000093, 20000094 → 170985
C	1	20000229, 20000230, 20000231 → 230735
D, E, F	0	–

Example (A, 6 Sep 2026): 998 → 8917 at 10:00:00, 200,000 RWF; 8917 → 16836 at 10:00:40, 196,000 RWF (98 %); 16836 → 24755 at 10:01:20, 192,080 RWF (98 %).



Not proof that the same funds moved: wallets hold pooled balances; 8917 may already have had 196,000 RWF and paid a debt that was coincidentally due. Timing and amount similarity are consistent with pass-through but do not trace money.

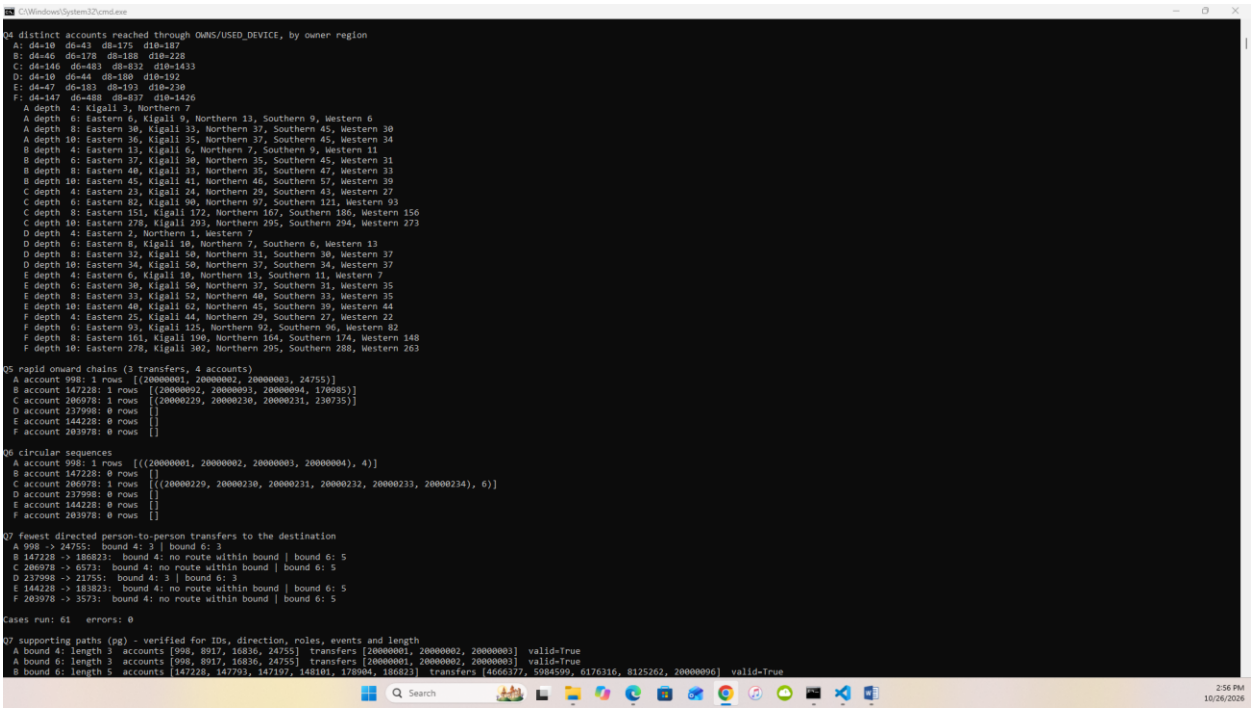


5.2

5.3 Q6 Circular payments

Set	Cycles	Sequence (length)
A	1	20000001 → 20000002 → 20000003 → 20000004 (4) – returns to 998 at 10:02:00, 188,238 RWF, span 120 s
C	1	20000229 … 20000234 (6), span 200 s
B, D, E, F	0	–

Further evidence to request: KYC of the accounts (are 8917, 16836, 24755 related people?), the device used to *initiate* each payment and its location, balances before/after, the source of 998's 200,000 RWF and what 998 did next, fees charged, earlier months (one-off or recurring), and any customer explanation (e.g. a rotating savings group, *ikimina*).

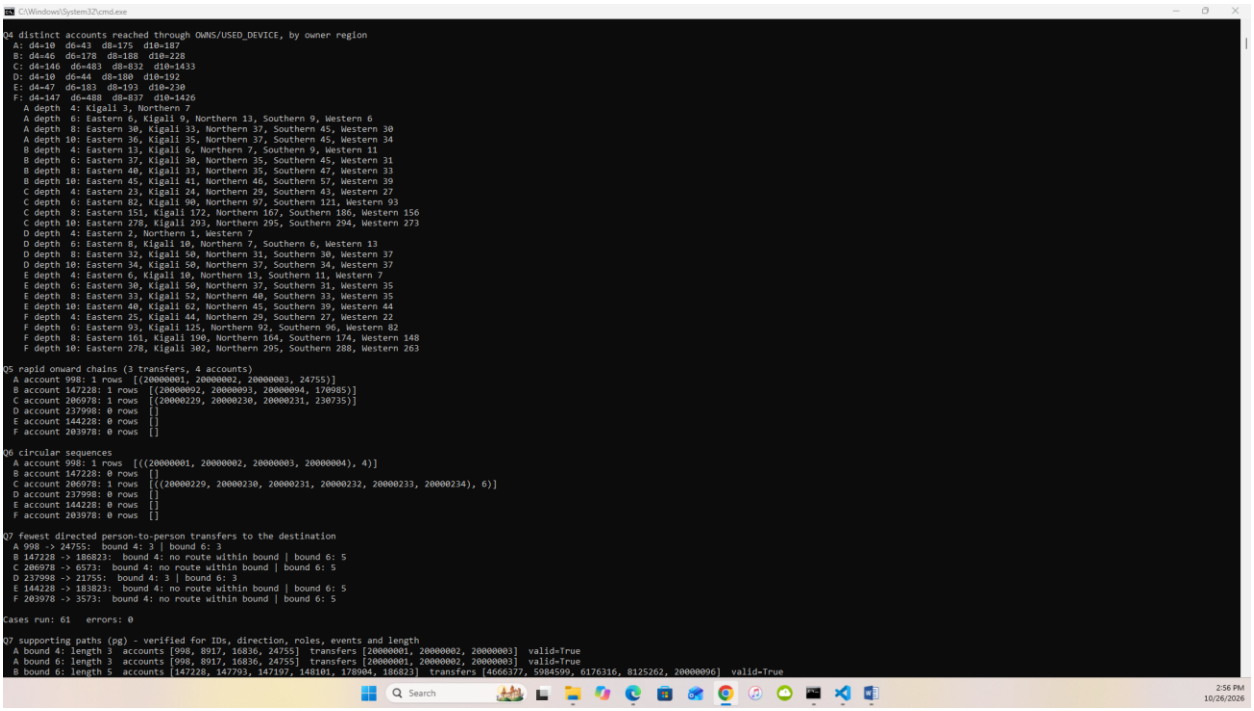


5.3

5.4 Q7 Shortest transfer connection (person accounts only)

Set	Review → destination	Bound 4	Bound 6	Verified path (bound 6)
A	998 → 24755	3	3	998→8917→16836→24755
B	147228 → 186823	no route	5	PG: 147228→147793→147197→148101→178904→186823; Neo4j: 147228→148132→147423→148101→178904→186823
C	206978 → 6573	no route	5	206978→214897→222816→230735→238654→6573
D	237998 → 21755	3	3	237998→5917→13836→21755
E	144228 → 183823	no route	5	144228→152147→…→183823
F	203978 → 3573	no route	5	203978→211897→…→3573

All paths verified for transfer IDs, direction, person roles, event existence and length (both engines). For B the two engines returned **different but equally short** paths – both valid – so paths are compared by validity, not equality. **No route within 4 ≠ no connection**: B, C, E, F all connect at 5; the bound, the person-only filter and the one-month window all hide longer or non-person routes (e.g. through an agent). **A route is not a flow of funds**: Q7 ignores time order and amounts – the teaching example's upper route runs backwards in time, so money could not have travelled along it.



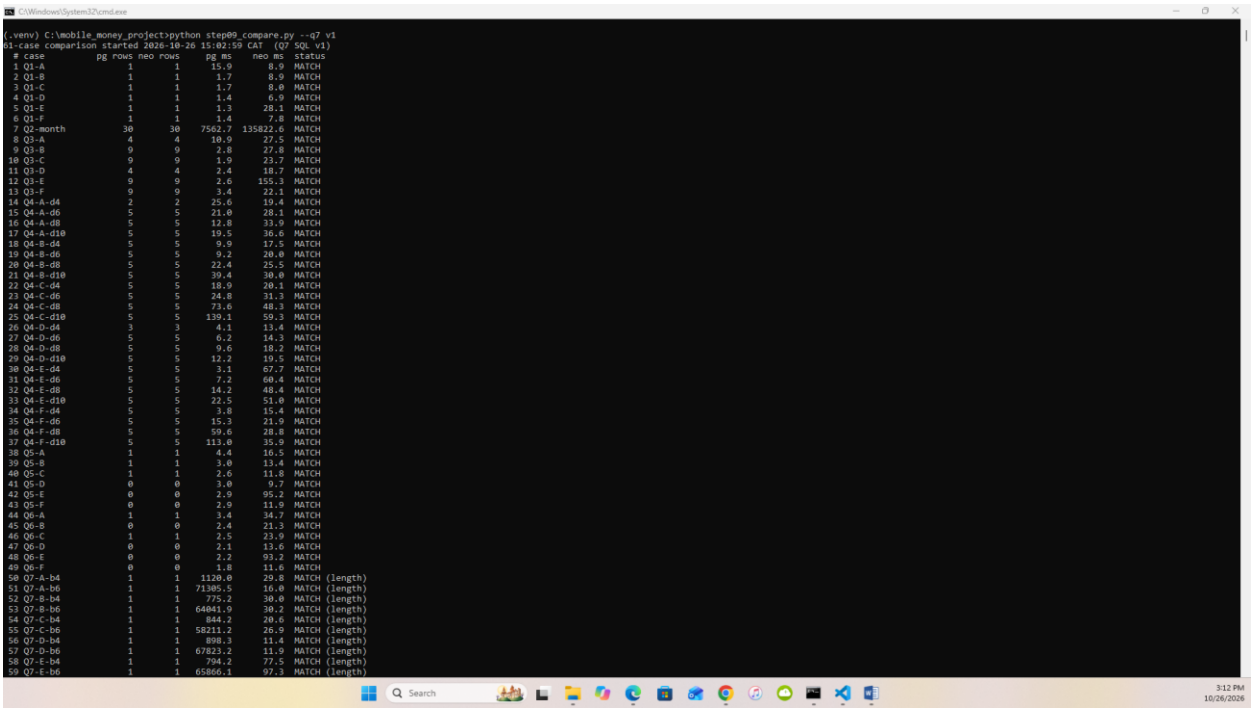
5.4

---

6 Correctness and performance

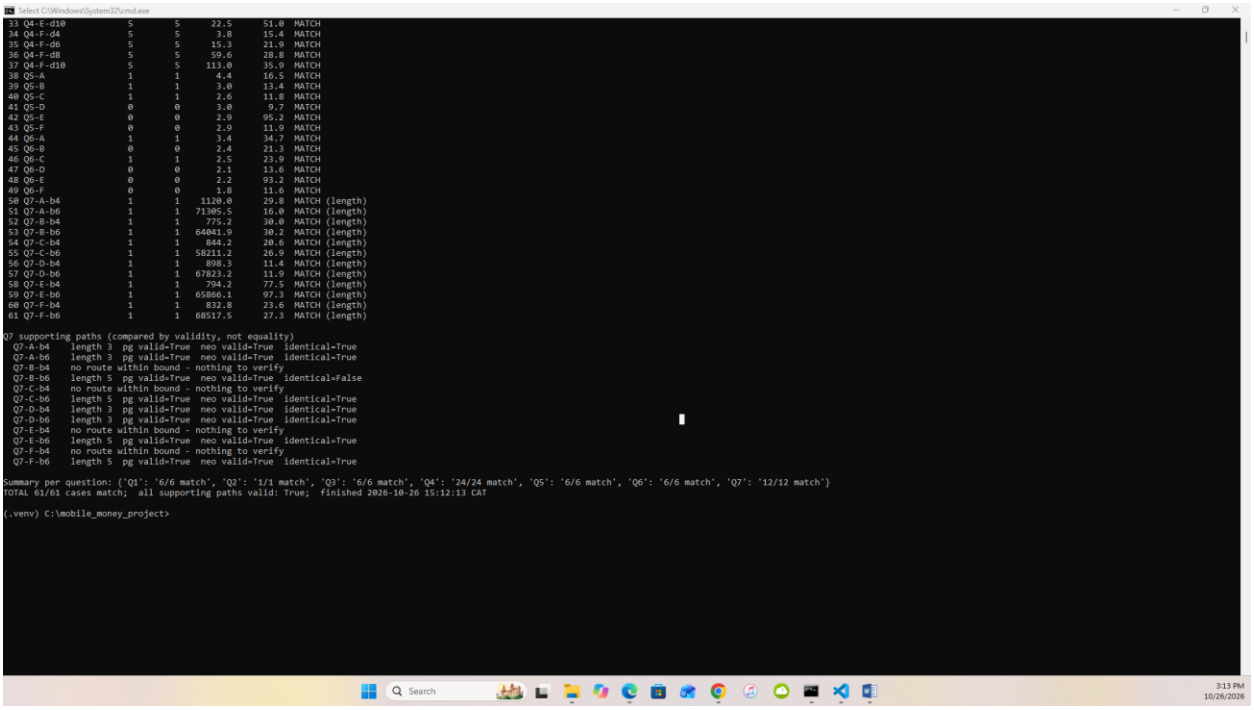
6.1 Comparison program

run_pg() / run_neo() return every row; rows are normalised (ints, tuples, ISO dates) and sorted before exact comparison. Result: **61 / 61 cases match** (Q1 6, Q2 1, Q3 6, Q4 24, Q5 6, Q6 6, Q7 12 lengths) and all 8 non-empty Q7 supporting paths are valid in both engines. No query result ever differed between the engines. Problems fixed on the way were all in tooling: the PostgreSQL check script did not recognise PostgreSQL 18's NOT NULL constraint type, a diagnostic Cypher line dropped a variable after WITH, and neo4j-admin needed an absolute --report-file path. Independent plain-Python check from the CSVs (step09b): the four set-A overlaps, the A chain and the A cycle equal both databases.



```
C:\Windows\System32\cmd.exe

(.venv) C:\mobile_money_project>python step09_compare.py --q7 v1
61-case comparison started 2026-10-26 15:02:59 CAT  (Q7 SQL v1)
  # case          pg rows neo rows     pg ms    neo ms  status
  1 Q1-A                1         1      15.9       8.9  MATCH
  2 Q1-B                1         1       1.7       8.9  MATCH
  3 Q1-C                1         1       1.7       8.0  MATCH
  4 Q1-D                1         1       1.4       6.9  MATCH
  5 Q1-E                1         1       1.3      28.1  MATCH
  6 Q1-F                1         1       1.4       7.8  MATCH
  7 Q2-month           30        30    7562.7  135822.6  MATCH
  8 Q3-A                4         4      10.9      27.5  MATCH
  9 Q3-B                9         9       2.8      27.8  MATCH
 10 Q3-C                9         9       1.9      23.7  MATCH
 11 Q3-D                4         4       2.4      18.7  MATCH
 12 Q3-E                9         9       2.6     155.3  MATCH
 13 Q3-F                9         9       3.4      22.1  MATCH
 14 Q4-A-d4             2         2      25.6      19.4  MATCH
 15 Q4-A-d6             5         5      21.0      28.1  MATCH
 16 Q4-A-d8             5         5      12.8      33.9  MATCH
 17 Q4-A-d10            5         5      19.5      36.6  MATCH
 18 Q4-B-d4             5         5       9.9      17.5  MATCH
 19 Q4-B-d6             5         5       9.2      20.0  MATCH
 20 Q4-B-d8             5         5      22.4      25.5  MATCH
 21 Q4-B-d10            5         5      39.4      30.0  MATCH
 22 Q4-C-d4             5         5      18.9      20.1  MATCH
 23 Q4-C-d6             5         5      24.8      31.3  MATCH
 24 Q4-C-d8             5         5      73.6      48.3  MATCH
 25 Q4-C-d10            5         5     139.1      59.3  MATCH
 26 Q4-D-d4             3         3       4.1      13.4  MATCH
 27 Q4-D-d6             5         5       6.2      14.3  MATCH
 28 Q4-D-d8             5         5       9.6      18.2  MATCH
 29 Q4-D-d10            5         5      12.2      19.5  MATCH
 30 Q4-E-d4             5         5       3.1      67.7  MATCH
 31 Q4-E-d6             5         5       7.2      60.4  MATCH
 32 Q4-E-d8             5         5      14.2      48.4  MATCH
 33 Q4-E-d10            5         5      22.5      51.0  MATCH
 34 Q4-F-d4             5         5       3.8      15.4  MATCH
 35 Q4-F-d6             5         5      15.3      21.9  MATCH
 36 Q4-F-d8             5         5      59.6      28.8  MATCH
 37 Q4-F-d10            5         5     113.0      35.9  MATCH
 38 Q5-A                1         1       4.4      16.5  MATCH
 39 Q5-B                1         1       3.0      13.4  MATCH
 40 Q5-C                1         1       2.6      11.8  MATCH
 41 Q5-D                0         0       3.0       9.7  MATCH
 42 Q5-E                0         0       2.9      95.2  MATCH
 43 Q5-F                0         0       2.9      11.9  MATCH
 44 Q6-A                1         1       3.4      34.7  MATCH
 45 Q6-B                0         0       2.4      21.3  MATCH
 46 Q6-C                1         1       2.5      23.9  MATCH
 47 Q6-D                0         0       2.1      13.6  MATCH
 48 Q6-E                0         0       2.2      93.2  MATCH
 49 Q6-F                0         0       1.8      11.6  MATCH
 50 Q7-A-b4             1         1    1120.0      29.8  MATCH (length)
 51 Q7-A-b6             1         1   71305.5      16.0  MATCH (length)
 52 Q7-B-b4             1         1     775.2      30.0  MATCH (length)
 53 Q7-B-b6             1         1   64041.9      30.2  MATCH (length)
 54 Q7-C-b4             1         1     844.2      20.6  MATCH (length)
 55 Q7-C-b6             1         1   58211.2      26.9  MATCH (length)
 56 Q7-D-b4             1         1     898.3      11.4  MATCH (length)
 57 Q7-D-b6             1         1   67823.2      11.9  MATCH (length)
 58 Q7-E-b4             1         1     794.2      77.5  MATCH (length)
 59 Q7-E-b6             1         1   65866.1      97.3  MATCH (length)
```



```
Select C:\Windows\System32\cmd.exe

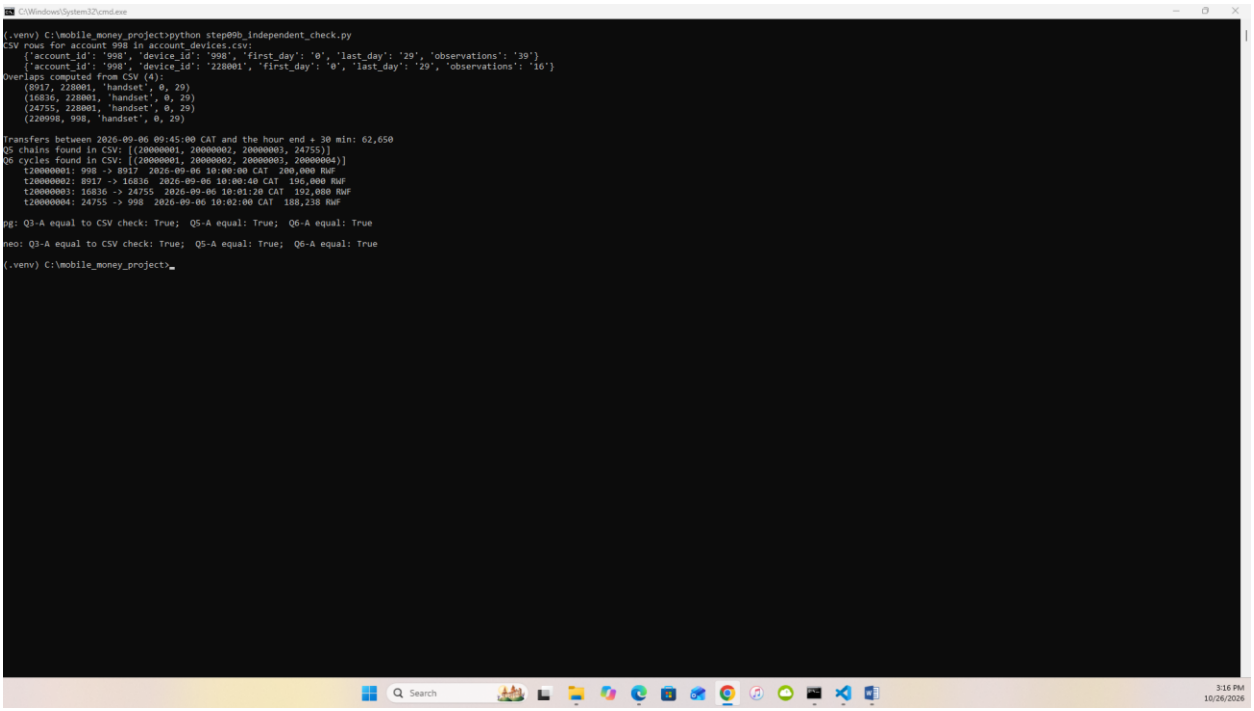
 33 Q4-E-d10            5         5      22.5      51.0  MATCH
 34 Q4-F-d4             5         5       3.8      15.4  MATCH
 35 Q4-F-d6             5         5      15.3      21.9  MATCH
 36 Q4-F-d8             5         5      59.6      28.8  MATCH
 37 Q4-F-d10            5         5     113.0      35.9  MATCH
 38 Q5-A                1         1       4.4      16.5  MATCH
 39 Q5-B                1         1       3.0      13.4  MATCH
 40 Q5-C                1         1       2.6      11.8  MATCH
 41 Q5-D                0         0       3.0       9.7  MATCH
 42 Q5-E                0         0       2.9      95.2  MATCH
 43 Q5-F                0         0       2.9      11.9  MATCH
 44 Q6-A                1         1       3.4      34.7  MATCH
 45 Q6-B                0         0       2.4      21.3  MATCH
 46 Q6-C                1         1       2.5      23.9  MATCH
 47 Q6-D                0         0       2.1      13.6  MATCH
 48 Q6-E                0         0       2.2      93.2  MATCH
 49 Q6-F                0         0       1.8      11.6  MATCH
 50 Q7-A-b4             1         1    1120.0      29.8  MATCH (length)
 51 Q7-A-b6             1         1   71305.5      16.0  MATCH (length)
 52 Q7-B-b4             1         1     775.2      30.0  MATCH (length)
 53 Q7-B-b6             1         1   64041.9      30.2  MATCH (length)
 54 Q7-C-b4             1         1     844.2      20.6  MATCH (length)
 55 Q7-C-b6             1         1   58211.2      26.9  MATCH (length)
 56 Q7-D-b4             1         1     898.3      11.4  MATCH (length)
 57 Q7-D-b6             1         1   67823.2      11.9  MATCH (length)
 58 Q7-E-b4             1         1     794.2      77.5  MATCH (length)
 59 Q7-E-b6             1         1   65866.1      97.3  MATCH (length)
 60 Q7-F-b4             1         1     832.8      23.6  MATCH (length)
 61 Q7-F-b6             1         1   68517.5      27.3  MATCH (length)

Q7 supporting paths (compared by validity, not equality)
  Q7-A-b4    length 3  pg valid=True  neo valid=True  identical=True
  Q7-A-b6    length 3  pg valid=True  neo valid=True  identical=True
  Q7-B-b4    no route within bound - nothing to verify
  Q7-B-b6    length 5  pg valid=True  neo valid=True  identical=False
  Q7-C-b4    no route within bound - nothing to verify
  Q7-C-b6    length 5  pg valid=True  neo valid=True  identical=True
  Q7-D-b4    length 3  pg valid=True  neo valid=True  identical=True
  Q7-D-b6    length 3  pg valid=True  neo valid=True  identical=True
  Q7-E-b4    no route within bound - nothing to verify
  Q7-E-b6    length 5  pg valid=True  neo valid=True  identical=True
  Q7-F-b4    no route within bound - nothing to verify
  Q7-F-b6    length 5  pg valid=True  neo valid=True  identical=True

Summary per question: {'Q1': '6/6 match', 'Q2': '1/1 match', 'Q3': '6/6 match', 'Q4': '24/24 match', 'Q5': '6/6 match', 'Q6': '6/6 match', 'Q7': '12/12 match'}
TOTAL 61/61 cases match;  all supporting paths valid: True;  finished 2026-10-26 15:12:13 CAT

(.venv) C:\mobile_money_project>
```

6.1a table



6.1b independent check

6.2 Timings (3 warm-ups, 5 measured runs, one engine at a time)

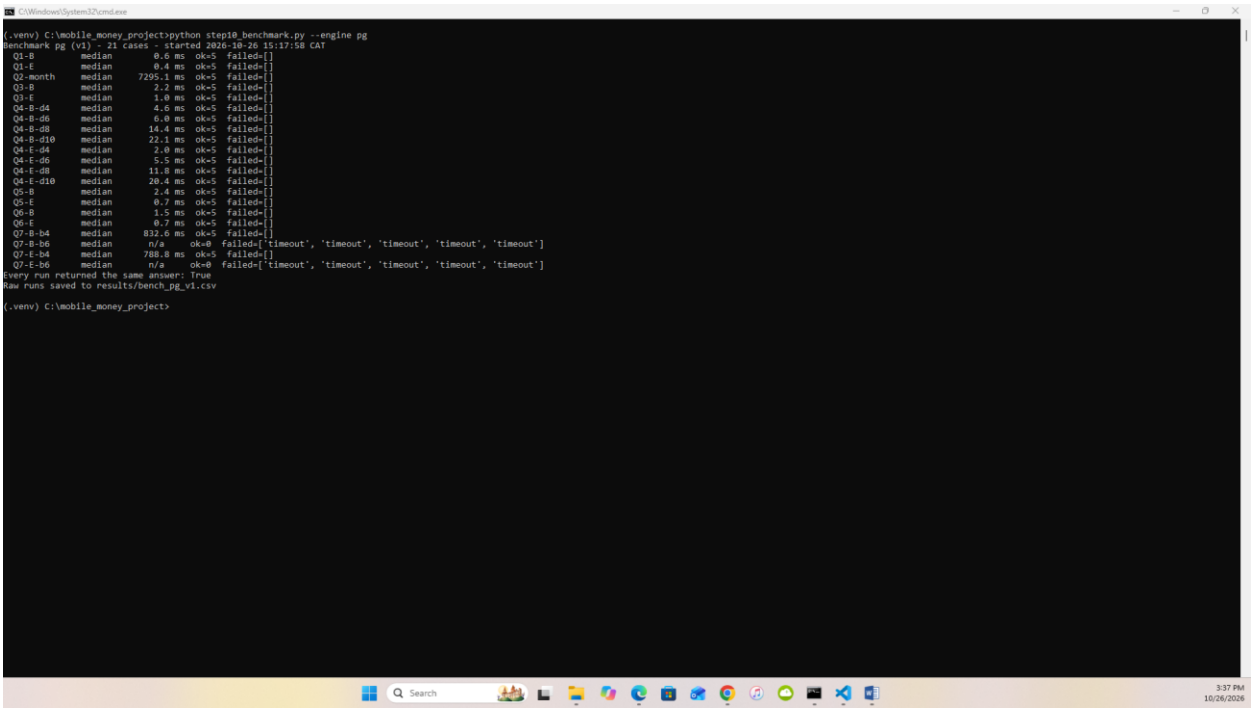
Medians, with min–max in brackets, in milliseconds. Ratio = PostgreSQL ÷ Neo4j (>1 means Neo4j faster).

Case	PostgreSQL	Neo4j	PG/Neo4j
Q1 B	0.6 (0.6–0.8)	9.7 (8.5–11.9)	0.07
Q1 E	0.4 (0.4–0.7)	9.3 (8.1–10.2)	0.05
Q2 month	**7,295** (7,055–7,341)	**timeout 5/5** (>60 s; one untimed PROFILE run 137 s)	n/a – PG >8× faster
Q3 B	2.2 (1.8–2.4)	10.4 (9.1–11.2)	0.21
Q3 E	1.0 (1.0–1.1)	9.5 (8.6–9.6)	0.11
Q4 B d4 / d6 / d8 / d10	4.6 / 6.0 / 14.4 / **22.1**	10.7 / 12.2 / 15.2 / **11.9**	0.43 / 0.49 / 0.94 / **1.85**
Q4 E d4 / d6 / d8 / d10	2.0 / 5.5 / 11.8 / **20.4**	8.1 / 10.1 / 9.9 / **10.7**	0.24 / 0.55 / 1.19 / **1.91**
Q5 B	2.4 (1.7–2.6)	7.9 (7.4–8.5)	0.31
Q5 E	0.7 (0.6–0.9)	6.7 (6.5–7.2)	0.11
Q6 B	1.5 (1.2–1.7)	18.3 (16.6–18.5)	0.08
Q6 E	0.7 (0.6–0.8)	7.8 (7.0–8.0)	0.09
Q7 B bound 4, SQL v1	832.6 (810–840)	23.4 (21.3–28.1)	35.6
Q7 B bound 6, SQL v1	**timeout 5/5** (>60 s)	27.1 (25.5–28.7)	n/a

Case	PostgreSQL	Neo4j	PG/Neo4j
Q7 E bound 4, SQL v1	788.8 (787–802)	21.3 (20.7–26.3)	37.1
Q7 E bound 6, SQL v1	**timeout 5/5** (>60 s)	35.6 (33.1–36.5)	n/a
Q7 B bound 4 / 6, SQL v2 (§6.3)	*13.4 / 128.1*	*23.4 / 27.1*	*0.57 / 4.73*
Q7 E bound 4 / 6, SQL v2 (§6.3)	*12.1 / 125.7*	*21.3 / 35.6*	*0.57 / 3.53*

Errors and timeouts (recorded separately, never as timings): PostgreSQL Q7 bound 6 with SQL v1, 5/5 timeouts for both B and E; Neo4j Q2, 5/5 timeouts. No errors. Every completed run of a case returned the same answer. Raw runs: results/bench_pg_v1.csv, bench_pg_v2_Q7.csv, bench_neo_cypher.csv. **Measurement notes:** the Neo4j service was idle (not stopped) during the PostgreSQL runs; Neo4j's ≈7–10 ms floor includes opening a driver session per call in our run_neo() wrapper, which inflates its small-query times.

Processing differences (plans in results/plans/). *Q4 B depth 10.* PostgreSQL's recursive CTE probed four B-tree indexes (accounts by customer and by id, account_devices in both directions) once per frontier node: 958 nodes, **14,288 buffer hits, 40 ms**. Neo4j used VarLengthExpand(Pruning,BFS,All) from one NodeUniqueIndexSeek, following stored relationship pointers breadth-first without revisiting nodes: 453 nodes reached, **1,885 db hits** for the expansion (3,708 total). Neo4j's cost depends on the neighbourhood, not on depth, which is why its time stays flat from d4 to d10 while PostgreSQL's rises. *Q2 month.* PostgreSQL ran a **parallel sequential scan with 4 workers** and a two-stage hash aggregate over 147,139 pages (7.4 s). Neo4j Community's SLOTTED runtime is single-threaded: a DirectedRelationshipTypeScan over all 20,003,300 TRANSFER relationships, two property reads and a datetime construction per row – **100 million db hits, 137 s**.



6.2

6.3 SQL improvement for Q7

Wasted work in v1 (EXPLAIN ANALYZE, B bound 6, our machine – 75.7 s): the recursive walk produced **427,955** (account, depth) rows, made **187,957** index scans of transfers_sender_time_idx returning **14.57 M** transfer rows, then **14.57 M** accounts role look-ups – **60.1 M buffer hits** – to find a destination 5 steps away. It cannot stop when the destination is found and re-expands accounts already reached at a smaller depth.

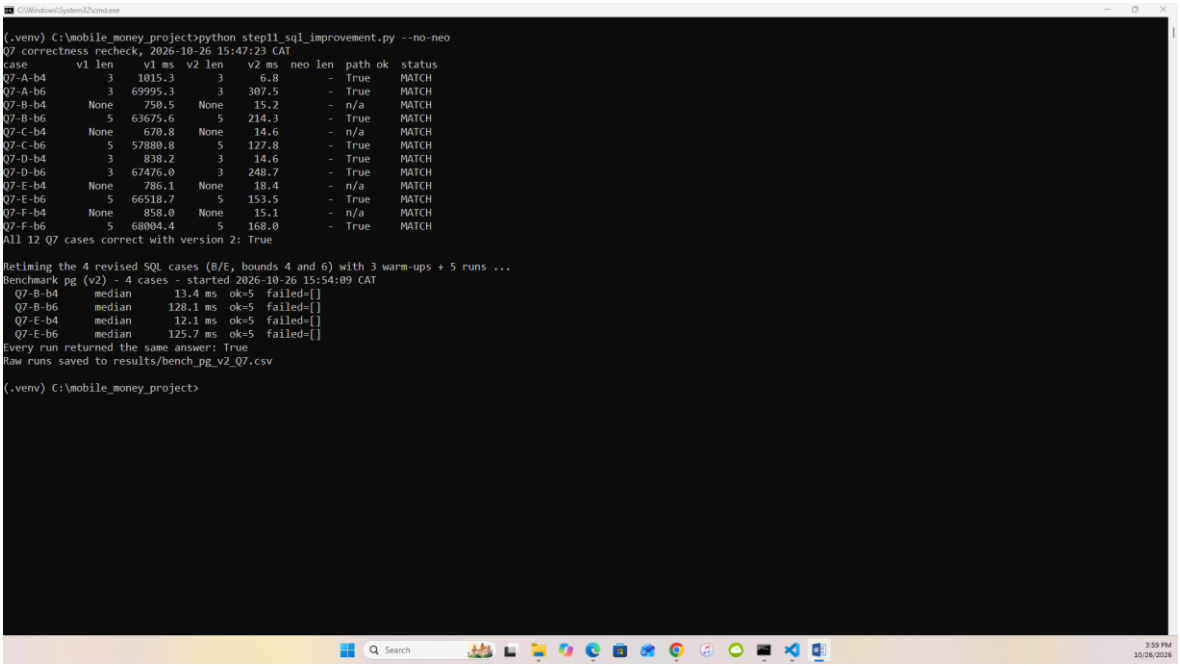
Prediction (before running v2): searching 3 levels forward from the review account and 3 backward from the destination (Q7_SQL_V2, meet in the middle) touches about $16^3 \approx 4{,}000$ states per side instead of $\approx 16^5$, so it should be two to three orders of magnitude faster with identical lengths.

Result: all **12** lengths unchanged (6 × "3 or 5", 4 × "no route within 4"), all supporting paths valid. Plan after (B bound 6): forward **2,216** rows from 188 index scans, backward **2,560** rows from 210 index scans, merge join with 26 meeting points – **111,730 buffer hits (540× fewer), 158.7 ms** under EXPLAIN ANALYZE.

Case	v1 median	v2 median	Speed-up
B bound 4	832.6 ms	13.4 ms	**62×**
B bound 6	timeout 5/5 (>60 s; one untimed run 63.7 s)	128.1 ms	**>470×** (≈500× vs 63.7 s)
E bound 4	788.8 ms	12.1 ms	**65×**

Case	v1 median	v2 median	Speed-up
E bound 6	timeout 5/5 (>60 s; one untimed run 66.5 s)	125.7 ms	**>470×** (≈530× vs 66.5 s)

Version 2 is selected for Section 7 and the final comparison. The gain is algorithmic (bidirectional search), not a property of the relational model – the same trick is what makes native shortest-path search fast.



6.3

---

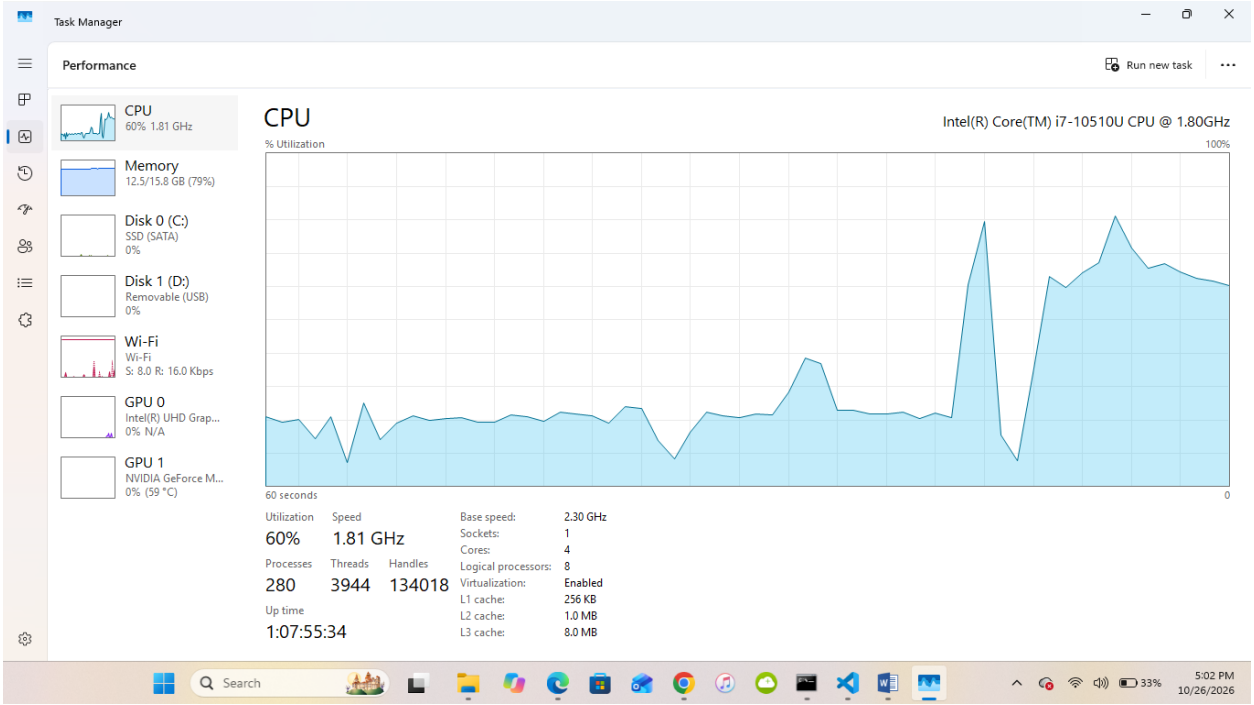
7 Concurrent requests

7.1 Load test

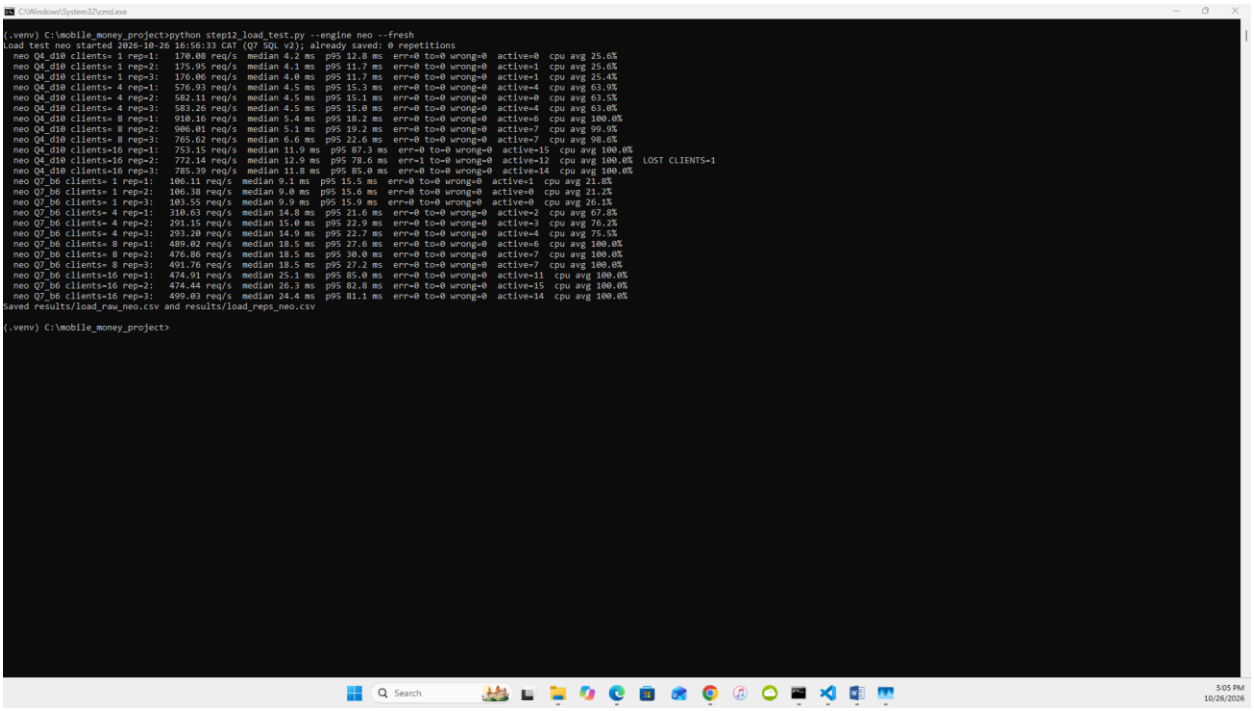
16 combinations (engine × workload × clients), median of 3 repetitions:

Engine	Workload	Clients	Throughput req/s	Median ms	p95 ms	Errors	Timeouts
PostgreSQL	Q4 d10	1 / 4 / 8 / 16	28 / 79 / 129 / 129	20.0 / 27.5 / 35.9 / 96.8	87.8 / 135.0 / 143.7 / 334.6	0	0
PostgreSQL	Q7 b6 (v2)	1 / 4 / 8 / 16	8.1 / 21.8 / 34.0 / 33.3	126.1 / 187.6 / 234.1 / 424.9	155.2 / 243.0 / 283.6 / 811.9	0	0
Neo4j	Q4 d10	1 / 4 / 8 / 16	176 / 582 / 906 / 772	4.1 / 4.5 / 5.4 / 11.9	11.7 / 15.1 / 19.2 / 85.0	0 / 0 / 0 / 0 (1 lost client in one rep)	0
Neo4j	Q7 b6	1 / 4 / 8 / 16	106 / 293 / 489 / 475	9.1 / 14.9 / 18.5 / 25.1	15.6 / 22.7 / 27.6 / 82.8	0	0

No wrong answers in any repetition. In Neo4j Q4 with 16 clients (repetition 2) one of the 16 client processes failed to finish warming up while the CPU was saturated; it is counted as 1 error and the other two repetitions are complete. Each test used one persistent connection per process, a common start time and a 15-second deadline; active sessions were sampled from a separate monitoring connection.

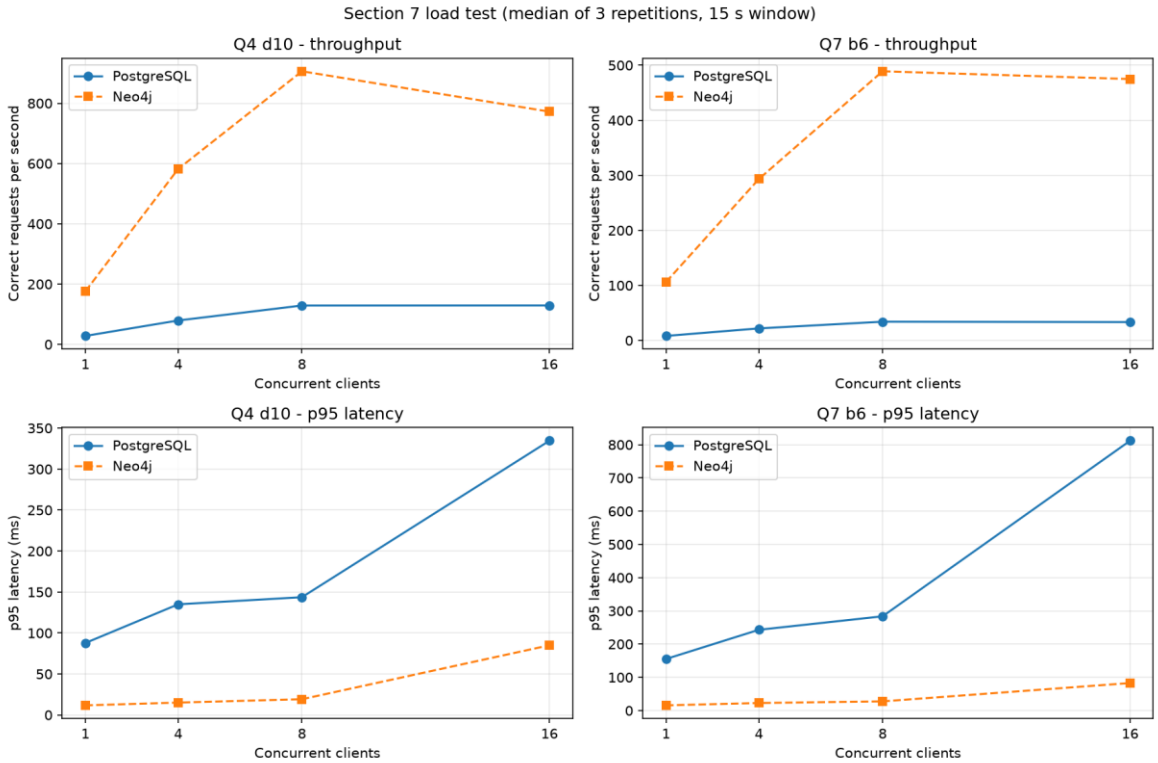


7.1a active clients / Task Manager



7.1b summary

7.2 Scaling



7.2 – results/plots/load_scaling.png, both engines

Neo4j. Throughput rose almost linearly from 1 to 8 clients (Q4: 176 → 906 req/s; Q7: 106 → 489 req/s) while median latency barely moved (Q4 4.1 → 5.4 ms). At 8 clients CPU reached ≈100 % of the 4 cores / 8 threads; beyond that point extra clients only queue: at 16 clients throughput fell slightly (Q4 772, Q7 475 req/s) and **p95 latency jumped about 4×** (Q4 19 → 85 ms, Q7 28 → 83 ms). Saturation point: about 8 clients.

PostgreSQL. The same shape at a lower level. Q4 throughput rose 28 → 79 → 129 req/s from 1 to 8 clients and then stayed flat (129 req/s at 16); Q7 (improved SQL v2) rose 8.1 → 21.8 → 34.0 and stayed at 33.3. CPU reached 100 % at 8 clients, so PostgreSQL also saturates at about 8 clients. After that point the extra clients only wait: from 8 to 16 clients median latency nearly tripled (Q4 35.9 → 96.8 ms, Q7 234 → 425 ms) and p95 rose 2.3× for Q4 (144 → 335 ms) and 2.9× for Q7 (284 → 812 ms). With one client PostgreSQL used only ≈28 % CPU – one query uses about one core – which is why adding clients up to the core count raised throughput.

Throughput vs latency. Throughput is the work the system completes per second; latency is what one analyst waits. Past saturation the system still finishes about the same number of requests per second, but each analyst waits longer, and the slowest 5 % wait much longer. Even with the improved SQL, Neo4j kept a large advantage under load: at 16 clients it served **6.0× more Q4** (772 vs 129 req/s) and **14.3× more Q7 requests per second** (475 vs 33), with lower p95 latency (Q4 85 vs 335 ms; Q7 83 vs 812 ms). The single-query gap (Q7 bound 6: 27–36 ms vs 126–128 ms) grows under concurrency because each SQL request burns more CPU, so PostgreSQL hits the CPU ceiling with fewer completed requests. No errors, timeouts or wrong answers occurred in any PostgreSQL repetition. These results hold only for this laptop, this one-month data size and these two workloads.

---

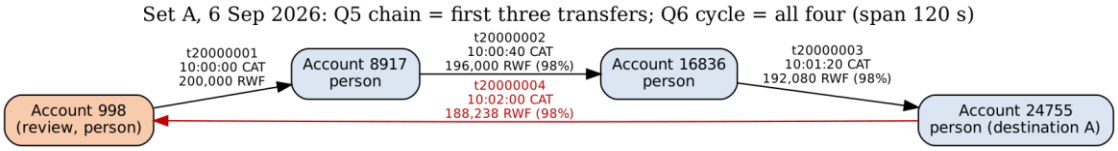
8 Decisions

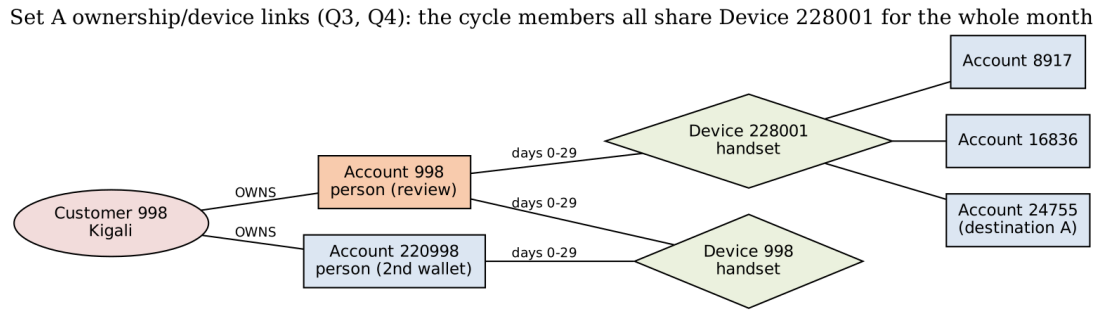
8.1 Investigation priorities

Pair	Q3 devices	Q4 reach d4→d10	Q5 chains	Q6 cycles	Q7 len	Review first
A 998 / D 237998	both share a scenario handset with 3 others incl. their destination	10→187 / 10→192	**1** / 0	**1** / 0	3 / 3	**A**
B 147228 / E 144228	both: household phone + scenario handset with 5 others incl. destination	46→228 / 47→230	**1** / 0	0 / 0	5 / 5	**B**
C 206978 / F 203978	both: household phone (replacement) + scenario handset with 5 others	146→1,433 / 147→1,426	**1** / 0	**1** / 0	5 / 5	**C**

Pair A vs D in detail. Device and network evidence is *identical in shape*: 998 shares handset 228001 with 8917, 16836 and 24755 for all 30 days, and 237998 shares handset 228301 with 5917, 13836 and 21755; both reach about 190 accounts by depth 10 and both connect to their

destination in 3 transfers. What separates them is **transfer behaviour**. On 6 Sep 998 sent 200,000 RWF at 10:00:00; within 40-second steps 196,000, 192,080 and 188,238 RWF moved 8917 → 16836 → 24755 → 998, each 98 % of the previous, returning to 998 120 s after it left (Q5 and Q6 both fire). D's group pays 45,000 RWF hops **two days apart** with no reduction, so neither rule fires. A is reviewed first.





Ordinary explanation. Four people who share one phone could be a household or a savings group (*ikimina*) passing a contribution round the circle; a 2 % reduction per hop could be fees or a small deduction. **Missing evidence:** who owns the shared handset, which device initiated each transfer, balances, KYC links between the four customers, and whether the pattern repeats. If KYC shows one family and the cycle recurs monthly with a fee schedule matching 2 %, priority would drop; if the four are unrelated and 998 immediately cashed out at an agent, it would rise. We do **not** declare fraud from these records.

8.2 Recommendation (≤ 500 words)

Reporting → PostgreSQL. Q1–Q2 are filters and aggregates over one large event table. With a covering B-tree, Q1 is sub-millisecond; Q2 is a single sequential scan and hash aggregate (**7.3 s** with 4 parallel workers) versus a Neo4j timeout (>60 s; 137 s untimed, single-threaded). Loading took 40.8 s plus 325.7 s for keys and indexes (≈6.1 min in total), against 77.8 s import plus 103.8 s constraint/index setup in Neo4j; storage 3.73 GB versus 2.05 GB. **Investigation → Neo4j for exploration, PostgreSQL acceptable for fixed rules.** Q4–Q7 are traversals. The Cypher is shorter and closer to the analyst's question – Q6 is one bounded path pattern against a 20-line recursive CTE, and Q7 is a single `SHORTEST 1` with the person filter inside the pattern. Neo4j answered Q4 depth 10 in ≈11–12 ms (PostgreSQL 20–22 ms) and Q7 bound 6 in 27–36 ms, where the first SQL version timed out after 60 s. **Evidence against our preference:** once the SQL was improved (bidirectional search) PostgreSQL answered Q7 bound 6 in ≈126–128 ms and bound 4 faster than Neo4j (12–13 ms vs 21–23 ms), and Q1/Q3/Q5/Q6 – anchored, time-bounded joins – took 0.4–2.4 ms in SQL against 7–18 ms in Neo4j. **Concurrency:** at 16 clients Neo4j served 772 Q4 and 475

Q7 requests per second against PostgreSQL's 129 and 33 – a 6× and 14× gap, since every traversal costs PostgreSQL more CPU and both engines saturate at about 8 clients. **Model vs implementation.** Graph modelling benefits: relationships are stored, so neighbourhood expansion needs no join planning and variable-length patterns are declarative. Implementation/ algorithm effects: v1 SQL was slow because of breadth-first re-expansion, not because of tables; Neo4j's shortest path is fast mainly because it runs a bidirectional BFS. Our SQL v2 shows the gap is largely algorithmic. **Freshness.** If operational data are copied into a graph, target ≤ 15 minutes lag (micro-batch CDC from PostgreSQL). Stale data would miss a cycle that completes in two minutes (case A) until the next batch, and a case could be closed on incomplete evidence. **Limits:** (1) one laptop, one month (20 M transfers), cold/warm cache effects and a single run series; (2) only seven question shapes – results may differ for global graph algorithms (community detection, PageRank) or for write-heavy workloads.

8.3 Individual judgement

Fulgence Muhirwa (101380)

The finding I trust most from this project is that the large Q7 gap between PostgreSQL and Neo4j was mainly an algorithm effect, not a data-model effect. Our first recursive SQL timed out after 60 seconds at bound 6 on every measured run; its plan showed 427,955 (account, depth) rows, 14.57 million transfer rows and 60 million buffer hits to find a destination five steps away. Neo4j answered the same question in 27–36 ms with a ShortestPath operator that searches from both ends. When we rewrote the SQL to search three steps forward and three backward and join in the middle, it returned identical lengths for all 12 cases in about 126–128 ms, roughly 500 times faster, and at bound 4 it even beat Neo4j (12–13 ms against 21–23 ms). This changes how I read any engine comparison: before crediting the database, check whether both sides run the same algorithm. A check that could overturn this: rerun with cold caches and with destinations exactly six steps away; if version 2's advantage shrinks sharply, part of the gain came from caching or from these particular cases.

Uwineza Deborah (101195)

My finding concerns Q4: reach through ownership and shared devices does not grow geometrically. Using the growth from depth 4 to depth 6, we predicted about 795 accounts at depth 10 for set A and 2,665 for set B; the real counts were 187 and 228. Only sets C and F kept growing, to about 1,430 accounts, and they are the only sets whose walks reach public agent terminals, from depth 3. The network is made of small household and scenario clusters joined by a few heavily shared devices. For an investigator this matters more than the raw

numbers: at depth 8–10 most “connected” accounts are linked through infrastructure such as an agent’s terminal, not through any personal relationship, so a large reach count is weak evidence on its own, and the depth limit should be chosen before looking at results. A check that could overturn this: rerun Q4 with public terminals excluded and count how many depth-10 accounts remain for C and F; if most remain, something else, such as replacement phones, drives the growth and my explanation is wrong.

Babatunde Oyedele (101385)

My finding comes from the load test: on our laptop both engines stopped scaling at about eight concurrent clients, which equals its eight logical CPU threads. Neo4j’s Q4 throughput rose from 176 to 906 requests per second between one and eight clients with almost no change in median latency, then fell slightly to 772 at sixteen clients while p95 latency jumped about four times, from 19 to 85 ms. PostgreSQL followed the same shape at a lower level, flattening at about 129 Q4 and 34 Q7 requests per second, with CPU at 100 % from eight clients. Even with the improved SQL, Neo4j served about 6 times more Q4 and 14 times more Q7 requests at sixteen clients. The lesson is that throughput and latency answer different questions: past saturation the system completes the same amount of work, but each analyst waits longer. A check that could overturn this: repeat the test with the client processes on a second computer or on a machine with more cores, because here clients and database shared the same four cores; if throughput keeps rising, the limit was our hardware, not the engines.

Contributions:

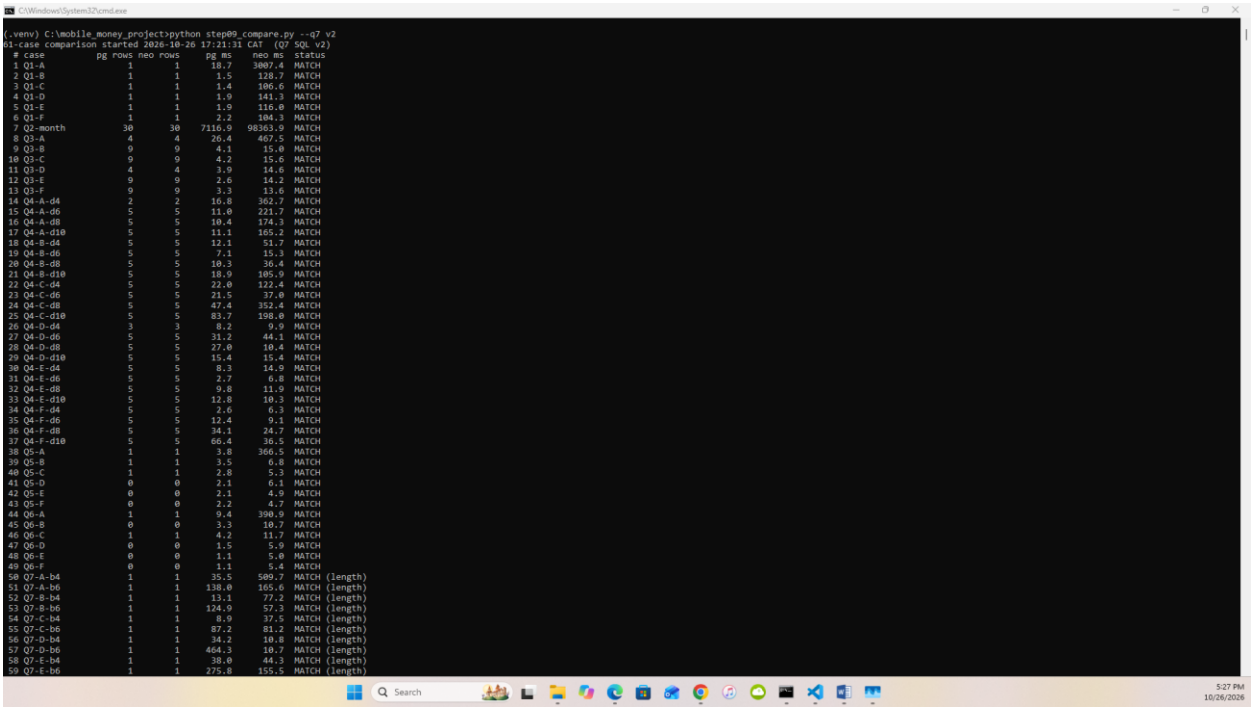
- **Fulgence Muhirwa (101380):** set up the environment on his laptop (PostgreSQL 18.6, Neo4j 2026.10.1 with Java 21, Python 3.12); ran every milestone end to end (capture, PostgreSQL load and checks, Neo4j import and checks, Q1–Q7 on both engines, the 61-case comparison, the benchmarks, the SQL improvement and the load test); fixed the problems found during the runs; assembled the report and the GitHub repository.
- **Uwineza Deborah (101195):** analysed the connected-account results (Q4–Q7), including the Q4 depth prediction against the actual counts, and reviewed sections 4–5 and the investigation priorities in 8.1.
- **Babatunde Oyedele (101385):** analysed the performance results (timings, query plans and load test, sections 6–7) and reviewed the recommendation in 8.2.

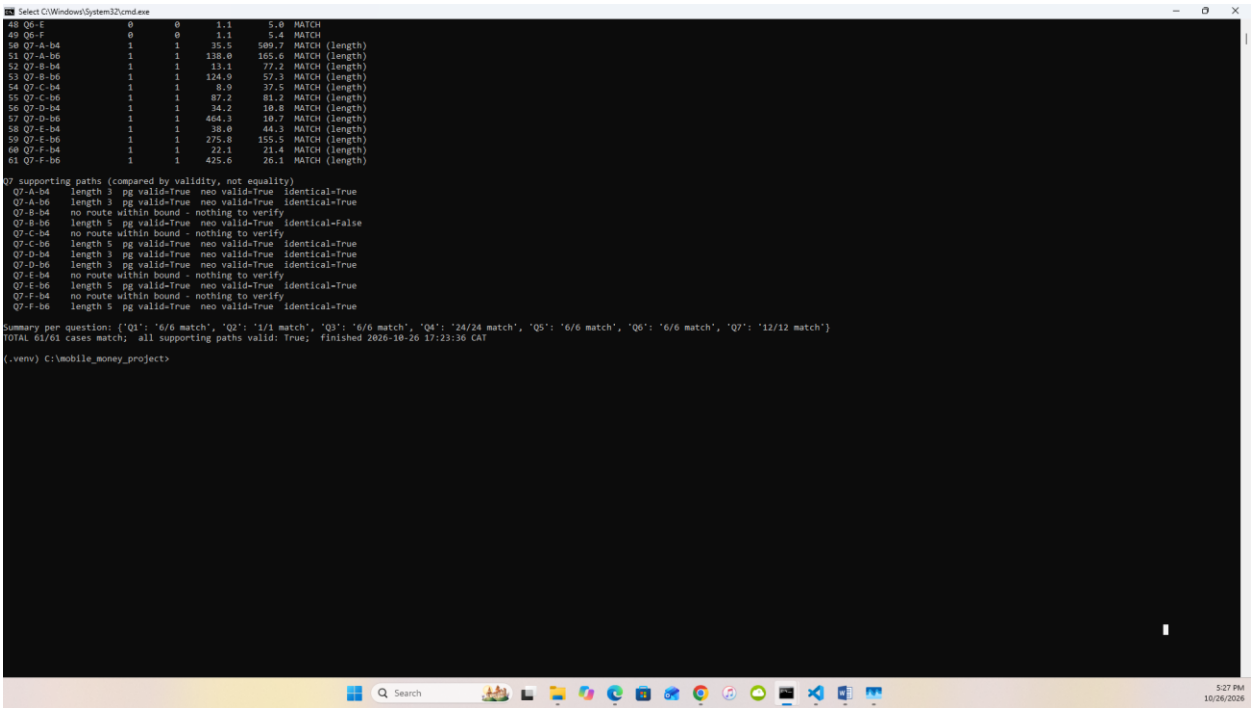
- **External/LLM assistance:**

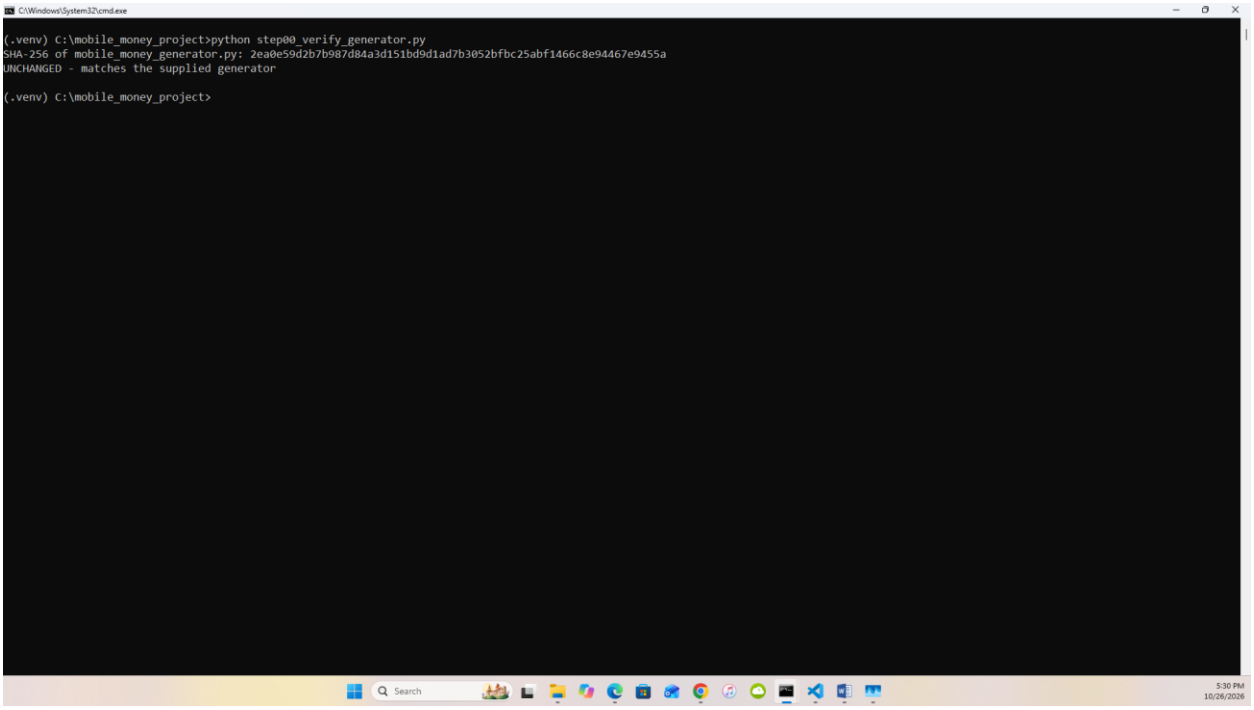
Claude (Anthropic) was used to draft code and text; every query result was checked by running both engines, the 61-case comparison and the independent CSV check, and we ran every step ourselves on our own machine instead of relying on the drafted code. Where it failed on our setup, we found and corrected the cause: PostgreSQL 18 listing NOT NULL constraints, Neo4j 2026.10 needing Java 21 and an absolute report-file path, and the load test's Windows multiprocessing. We compared each output with the expected answers, checked every number in the report against the saved script outputs in results/, and confirmed with a SHA-256 check that the supplied generator was unchanged.

9 Reproducibility

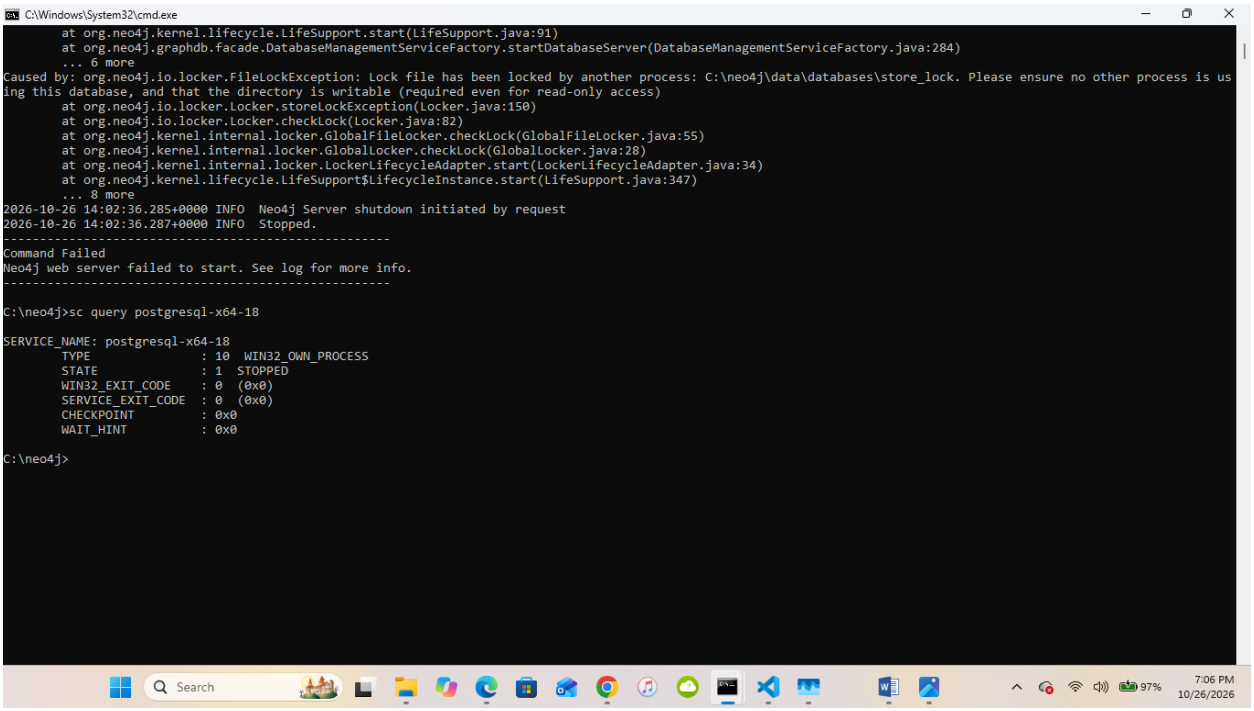
Code, raw results and plans: GitHub repository https://github.com/Mfulgence/Mobile_Money_Investigations_with_PostgreSQL_and_Neo4j and the ZIP. README lists commands in execution order; secrets are placeholders. step00_verify_generator.py confirms the generator is unchanged.







9.2a final 61-case rerun



9.2b services stopped